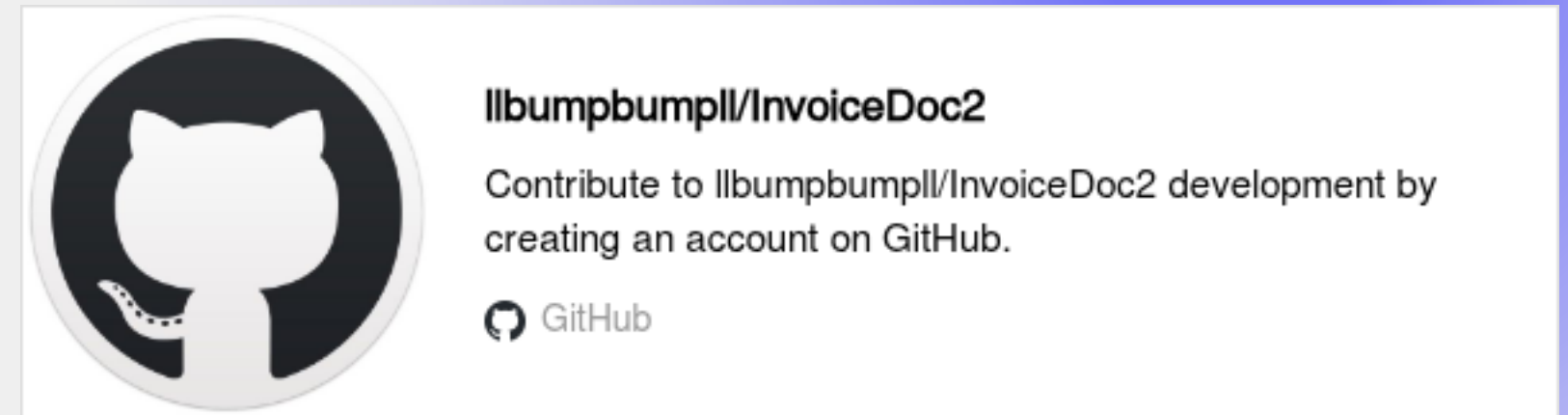


InvoiceDoc2

A full-stack Invoice Management System built with React, Express, and PostgreSQL.

GitHub Repo

GitHub

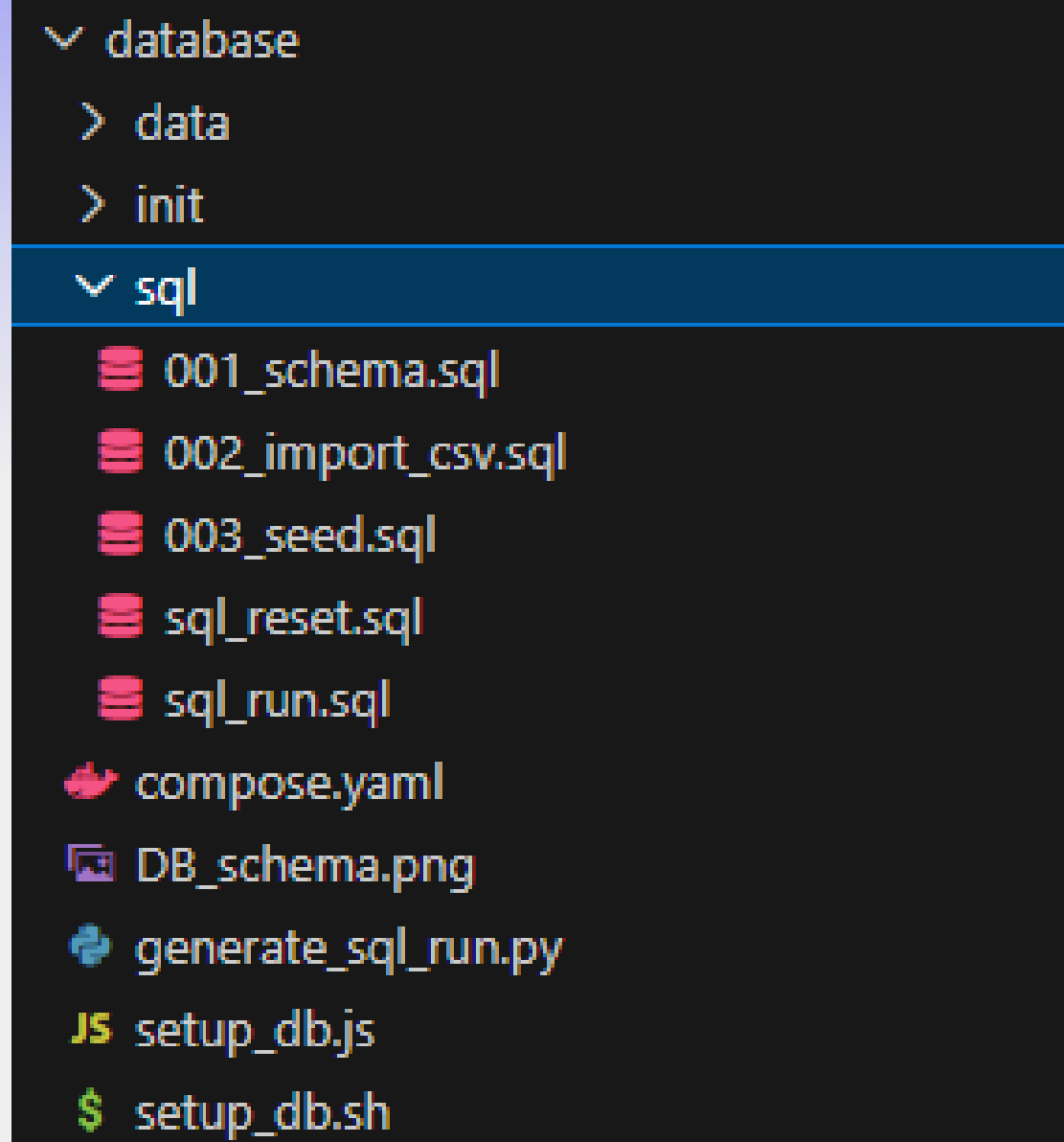


Part 4

Database

Important Files & Folder

Path: database/



This folder includes all files about database managing.

Ex.

- SQL Schema
- Seed Data
- Docker Compose
- Database Setup

Important Files & Folder

File: database/sql/001_schema.sql

001_schema.sql

- Create and manage table structure
- Contain all tables

Ex.

- country
- units
- customer
- product
- invoice
- invoice_line_item

```
001_schema.sql X
database > sql > 001_schema.sql
1  \set ON_ERROR_STOP on
2
3  -- Schema-only setup for InvoiceDoc2
4  -- Safe to run multiple times: creates missing tables and indexes only.
5
6  create table if not exists country (
7      id bigint primary key generated by default as identity,
8      created_at timestamp default now(),
9      code text unique not null,
10     name text not null
11 );
12
13 create table if not exists units (
14     id bigint primary key generated by default as identity,
15     created_at timestamp default now(),
16     code text unique not null,
17     name text not null
18 );
19
```

We use this file as a reference of data model.

Important Files & Folder

File: database/sql/003_seed.sql

```
003_seed.sql X
database > sql > 003_seed.sql
1  \set ON_ERROR_STOP on
2
3  -- Seed-only setup for InvoiceDoc2
4  -- Safe to rerun: inserts use ON CONFLICT DO NOTHING so partial seed repairs do not wipe user data.
5
6  -- =====
7
8  -- Populating table: country
9  INSERT INTO country (id, created_at, code, name) VALUES (1, '2026-01-12 17:19:44.412559+00', 'TH', 'Thailand') ON CONFLICT DO NOTHING;
10 INSERT INTO country (id, created_at, code, name) VALUES (3, '2026-01-12 17:20:38.972804+00', 'SG', 'Singapore') ON CONFLICT DO NOTHING;
11 INSERT INTO country (id, created_at, code, name) VALUES (5, '2026-01-12 17:48:45.61784+00', 'US', 'United States of America') ON CONFLICT D
```

003_seed.sql

- Collect data samples
- For example, information such as country, unit, customer, product, and sample invoice.

Important Files & Folder

File: database/sql/sql_run.sql

```
sql_run.sql X
database > sql > sql_run.sql
1  \set ON_ERROR_STOP on
2
3  -- Safe deploy script – creates missing tables and inserts seed data without dropping anything.
4  -- Safe to rerun on an existing database: uses IF NOT EXISTS and ON CONFLICT DO NOTHING.
5  -- For a full reset (drop + recreate), use sql_reset.sql instead.
6  -- Example usage: psql -f sql_run.sql
7  -- Database: PostgreSQL (Docker)
8
9  \ir 001_schema.sql
10 \ir 003_seed.sql
11 |
```

sql_run.sql

- Combine all schema and seed data together.
- Use for standard implementations. It creates a unique table that doesn't already exist (IF NOT EXISTS) and inserts unique initial data (ON CONFLICT DO NOTHING), allowing for reuse without data loss.

Important Files & Folder

File: database/sql/sql_reset.sql

```
108 -- =====
109 -- 2. Data Insertion
110 -- =====
111
112 -- Populating table: sales_person
113 INSERT INTO sales_person (id, created_at, code, name, start_work_date) VALUES (1, '2026-01-01 00:00:00+00', 'SP001', 'Somchai Jaidee', '2020-03-01');
114 INSERT INTO sales_person (id, created_at, code, name, start_work_date) VALUES (2, '2026-01-01 00:00:00+00', 'SP002', 'Nattaporn Srisuk', '2019-07-15');
115 INSERT INTO sales_person (id, created_at, code, name, start_work_date) VALUES (3, '2026-01-01 00:00:00+00', 'SP003', 'Apinya Wongsiri', '2021-01-10');
116 INSERT INTO sales_person (id, created_at, code, name, start_work_date) VALUES (4, '2026-01-01 00:00:00+00', 'SP004', 'Chaiwat Meechai', '2018-05-20');
117 INSERT INTO sales_person (id, created_at, code, name, start_work_date) VALUES (5, '2026-01-01 00:00:00+00', 'SP005', 'Pornpan Rattana', '2022-09-01');
118 INSERT INTO sales_person (id, created_at, code, name, start_work_date) VALUES (6, '2026-01-01 00:00:00+00', 'SP006', 'Wanchai Thammasat', '2017-11-30');
119 INSERT INTO sales_person (id, created_at, code, name, start_work_date) VALUES (7, '2026-01-01 00:00:00+00', 'SP007', 'Sukanya Prempree', '2023-02-14');
120 INSERT INTO sales_person (id, created_at, code, name, start_work_date) VALUES (8, '2026-01-01 00:00:00+00', 'SP008', 'Piyawat Khumdee', '2020-08-08');
121 INSERT INTO sales_person (id, created_at, code, name, start_work_date) VALUES (9, '2026-01-01 00:00:00+00', 'SP009', 'Malee Somboon', '2016-04-22');
122 INSERT INTO sales_person (id, created_at, code, name, start_work_date) VALUES (10, '2026-01-01 00:00:00+00', 'SP010', 'Thanachart Boonsri', '2024-06-01');
123
124 -- Populating table: country
125 INSERT INTO country (id, created_at, code, name) VALUES (1, '2026-01-12 17:19:44.412559+00', 'TH', 'Thailand');
126 INSERT INTO country (id, created_at, code, name) VALUES (3, '2026-01-12 17:20:38.972804+00', 'SG', 'Singapore');
127 INSERT INTO country (id, created_at, code, name) VALUES (5, '2026-01-12 17:48:45.61784+00', 'US', 'United States of America');
128
129 -- Populating table: units
130 INSERT INTO units (id, created_at, code, name) VALUES (1, '2026-01-12 17:50:21.26564+00', 'PCS', 'Pieces');
131 INSERT INTO units (id, created_at, code, name) VALUES (2, '2026-01-12 17:50:41.131446+00', 'MTS', 'Meters');
132 INSERT INTO units (id, created_at, code, name) VALUES (3, '2026-01-12 17:50:54.343991+00', 'KGS', 'Kilograms');
133 INSERT INTO units (id, created_at, code, name) VALUES (4, '2026-01-12 17:51:13.911009+00', 'PKT', 'Packets');
```

sql_reset.sql

```
sql_reset.sql X
database > sql > sql_reset.sql
1 -- RESET script -- drops ALL tables and recreates the database from scratch.
2 -- WARNING: ALL DATA WILL BE LOST.
3 -- Use this only for fresh installations or deliberate practice resets.
4 -- For upgrading an existing database, use the migration script instead:
5 -- database/sql/002_lab8_sales_person.sql
6 -- Example usage: psql -f sql_reset.sql
7 -- Database: PostgreSQL (Docker)
8
9 \set ON_ERROR_STOP on
10
11 -- =====
12 -- 1. Drop tables (child → parent order)
13 -- =====
14
15 drop table if exists invoice_line_item;
16 drop table if exists invoice;
17 drop table if exists sales_person;
18 drop table if exists product;
19 drop table if exists customer;
20 drop table if exists units;
21 drop table if exists country;
```

```
23 -- =====
24 -- 2. Schema Creation
25 -- =====
26
27 create table sales_person (
28     id bigint primary key generated by default as identity,
29     created_at timestamp default now(),
30     code text unique not null,
31     name text not null,
32     start_work_date date
33 );
34
35 create table country (
36     id bigint primary key generated by default as identity,
37     created_at timestamp default now(),
38     code text unique not null,
39     name text not null
40 );
```

- Use this only if you really need to **reset the database**. This will drop all tables and then create a new one with seed data.

Important Scripts

```
npm run db:reset
```

- This command runs the [sql_reset.sql](#) file
- Use for reset the database

Important!

Because of this command will run the [sql_run.sql](#) file that contain [schema](#) and [seed](#) data

- If you want to add tables, you should
 - Create new **migration file**
 - Create table in migration script file
- If you want to add seeds, you should
 - Create new **migration file**
 - Add seed data in migration script file

Always run the migration script after finished

Let's Try

Let's try adding payment table and seed data

Workflow

This is the step when you want to change or add new table and data to your database

- Open the database system – `npm run docker:db:start`
- Add tables
 - Create new **migration file**
 - Add new table to `migration file`
- Add seed data
 - Create new **migration file**
 - Add new seed data to `migration file`

Migration file store SQL commands for modifying the database structure, allowing the project to update the database sequentially and reducing the need to run the same commands repeatedly.

You can add both in the same migration file

1. Open the database system

For first time database opening

Run command in **root** terminal

npm run docker:db:start

```
PS E:\Work\InvoiceDoc2-learning> npm run docker:db:start

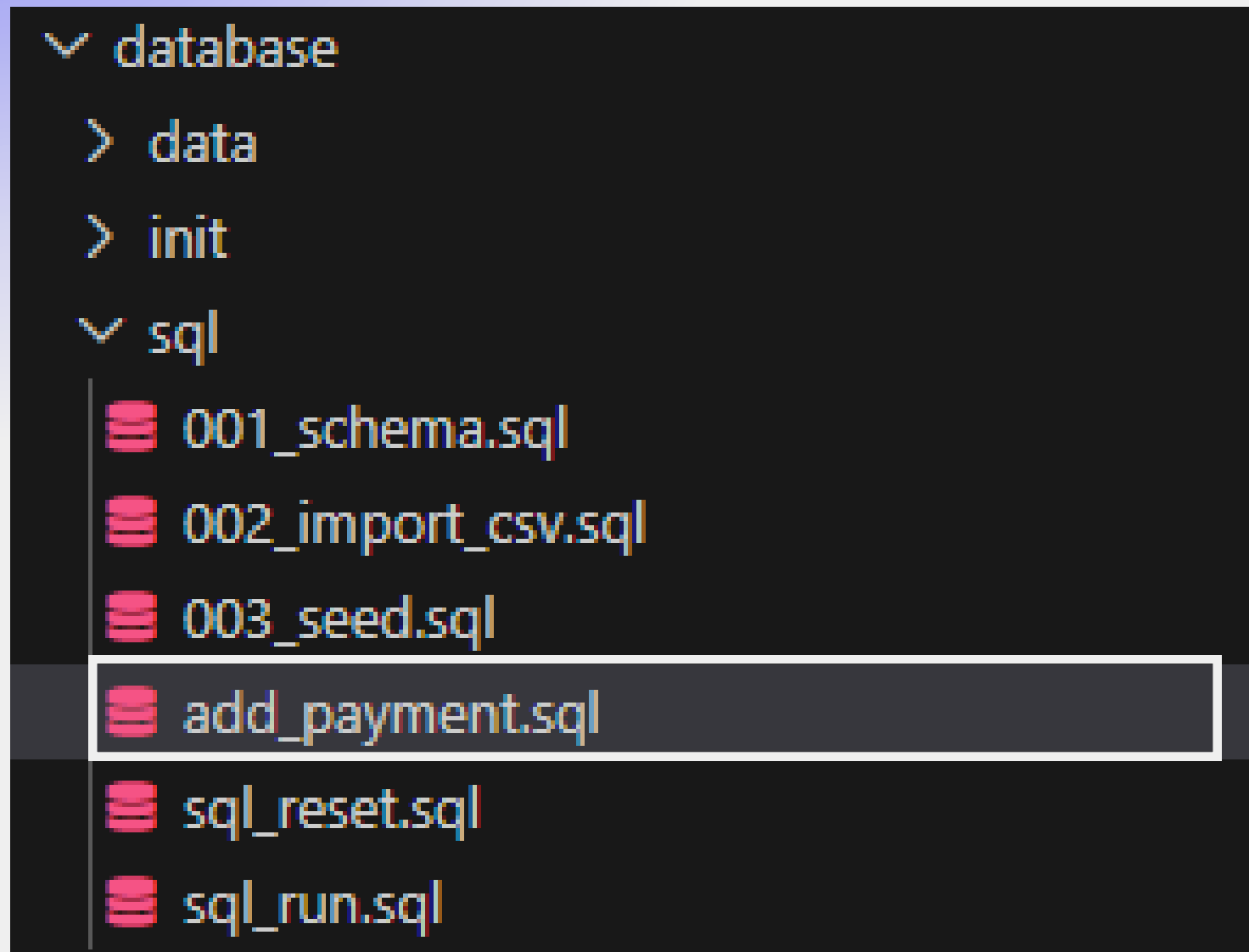
> invoicedoc2@2.0.0 docker:db:start
> node scripts/docker-db-start.js

🚀 Starting InvoiceDoc2 Database...
👉 Starting database and Adminer...
[+] up 2/2
  ✓ Container database-pgdatabase-1 Recreated 0.8s
  ✓ Container database-adminer-1 Running 0.0s
⌚ Waiting for database...
ERROR: Input redirection is not supported, exiting the process immediately.
Error: Command failed: timeout /t 5 /nobreak > nul
ERROR: Input redirection is not supported, exiting the process immediately.

    at genericNodeError (node:internal/errors:985:15)
    at wrappedFn (node:internal/errors:539:14)
    at checkExecSyncError (node:child_process:925:11)
    at execSync (node:child_process:997:15)
    at execShell (E:\Work\InvoiceDoc2-learning\scripts\run-safe.js:12:10)
    at main (E:\Work\InvoiceDoc2-learning\scripts\docker-db-start.js:63:14)
    at process.processTicksAndRejections (node:internal/process/task_queues:103:5) {
  status: 1,
  signal: null,
  output: [
    null,
    <Buffer >,
    <Buffer 45 52 52 4f 52 3a 20 49 6e 70 75 74 20 72 65 64 69 72 65 63 74 69 6f 6e 20 69 73 20 6e 6f 74 20 73 75 70 70
6f 72 74 65 64 2c 20 65 78 69 74 69 6e 67 ... 27 more bytes>
  ],
  pid: 28756,
  stdout: <Buffer >,
  stderr: <Buffer 45 52 52 4f 52 3a 20 49 6e 70 75 74 20 72 65 64 69 72 65 63 74 69 6f 6e 20 69 73 20 6e 6f 74 20 73 75
70 70 6f 72 74 65 64 2c 20 65 78 69 74 69 6e 67 ... 27 more bytes>
}
```

2. Create migration file

File: database/sql/add_payment.sql



The name of migration file is up to you.
Depends on the purpose of each migration.

Create new **migration file** in `database/sql/` path

3. Adding payment table

File: database/sql/add_payment.sql

add_payment.sql X

database > sql > add_payment.sql

```
1 create table if not exists payment (  
2     id bigint primary key generated by default as identity,  
3     invoice_id bigint not null references invoice(id) on delete cascade,  
4     payment_date date not null,  
5     amount numeric(12,2) not null,  
6     method text,  
7     note text  
8 );  
9
```

Create payment table in **migration file**

create table if not exists payment (
id bigint primary key generated by default as identity,
invoice_id bigint not null references invoice(id) on
delete cascade,
payment_date date not null,
amount numeric(12,2) not null,
method text,
note text
);

4. Adding seed data

File: database/sql/add_payment.sql

In this case, we create table and add seed data in the same migration.

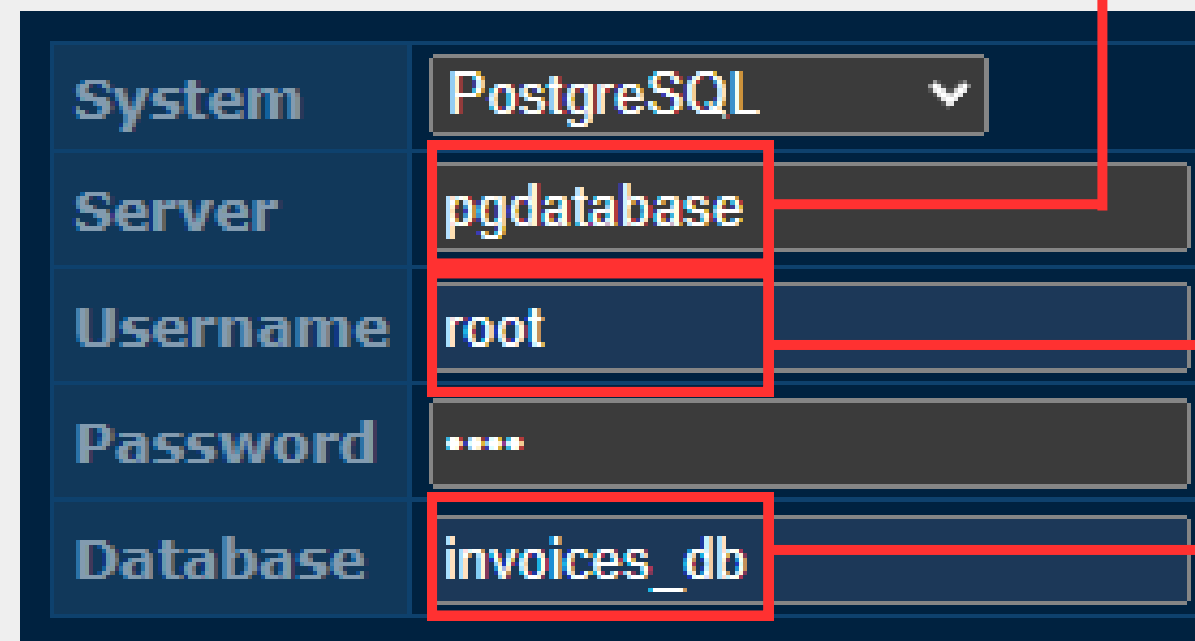
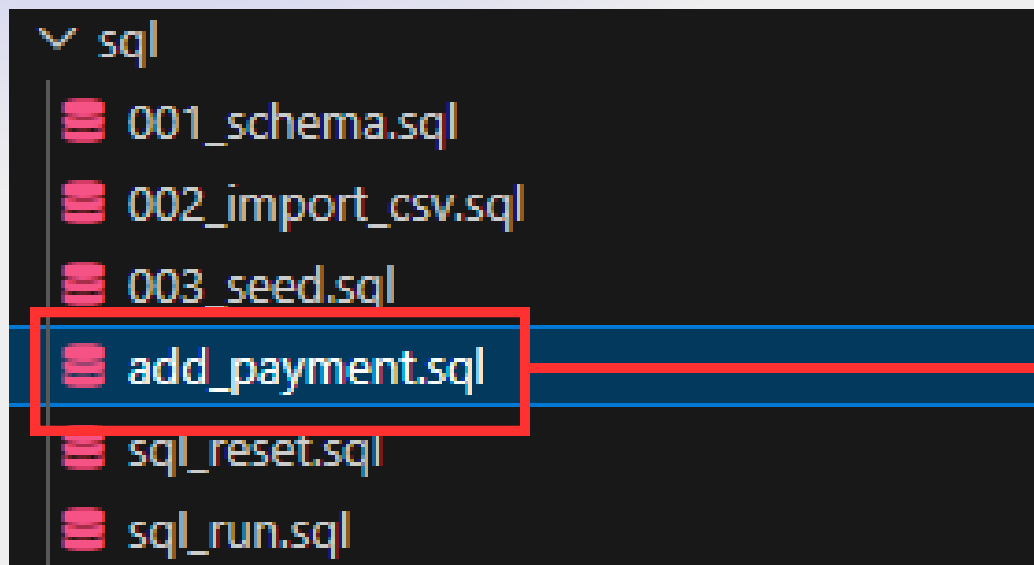
```
add_payment.sql X
database > sql > add_payment.sql
1  create table if not exists payment (
2      id bigint primary key generated by default as identity,
3      invoice_id bigint not null references invoice(id) on delete cascade,
4      payment_date date not null,
5      amount numeric(12,2) not null,
6      method text,
7      note text
8  );
9
10 INSERT INTO payment (id, invoice_id, payment_date, amount, method, note) VALUES (1, 2, '2026-03-01 00:00:00+00:00', 1500.00, 'bank_transfer', NULL) ON CONFLICT DO NOTHING;
11 INSERT INTO payment (id, invoice_id, payment_date, amount, method, note) VALUES (2, 3, '2026-03-02 01:00:00+00:00', 3200.00, 'cash', NULL) ON CONFLICT DO NOTHING;
```

```
INSERT INTO payment (id, invoice_id, payment_date, amount, method, note) VALUES (1, 2, '2026-03-01 00:00:00+00:00', 1500.00, 'bank_transfer', NULL) ON CONFLICT DO NOTHING;
INSERT INTO payment (id, invoice_id, payment_date, amount, method, note) VALUES (2, 3, '2026-03-02 01:00:00+00:00', 3200.00, 'cash', NULL) ON CONFLICT DO NOTHING;
```

5. Run the migration script

Run command in **root** terminal

`type database/sql/[migration name] | docker exec -i [postgres name] psql -U [user] -d [db name]`



```
PS E:\Work\InvoiceDoc2-main> type database/sql/add_payment.sql |
docker exec -i database-pgdatabase-1 psql -U root -d invoices_db
CREATE TABLE
INSERT 0 1
INSERT 0 1
```

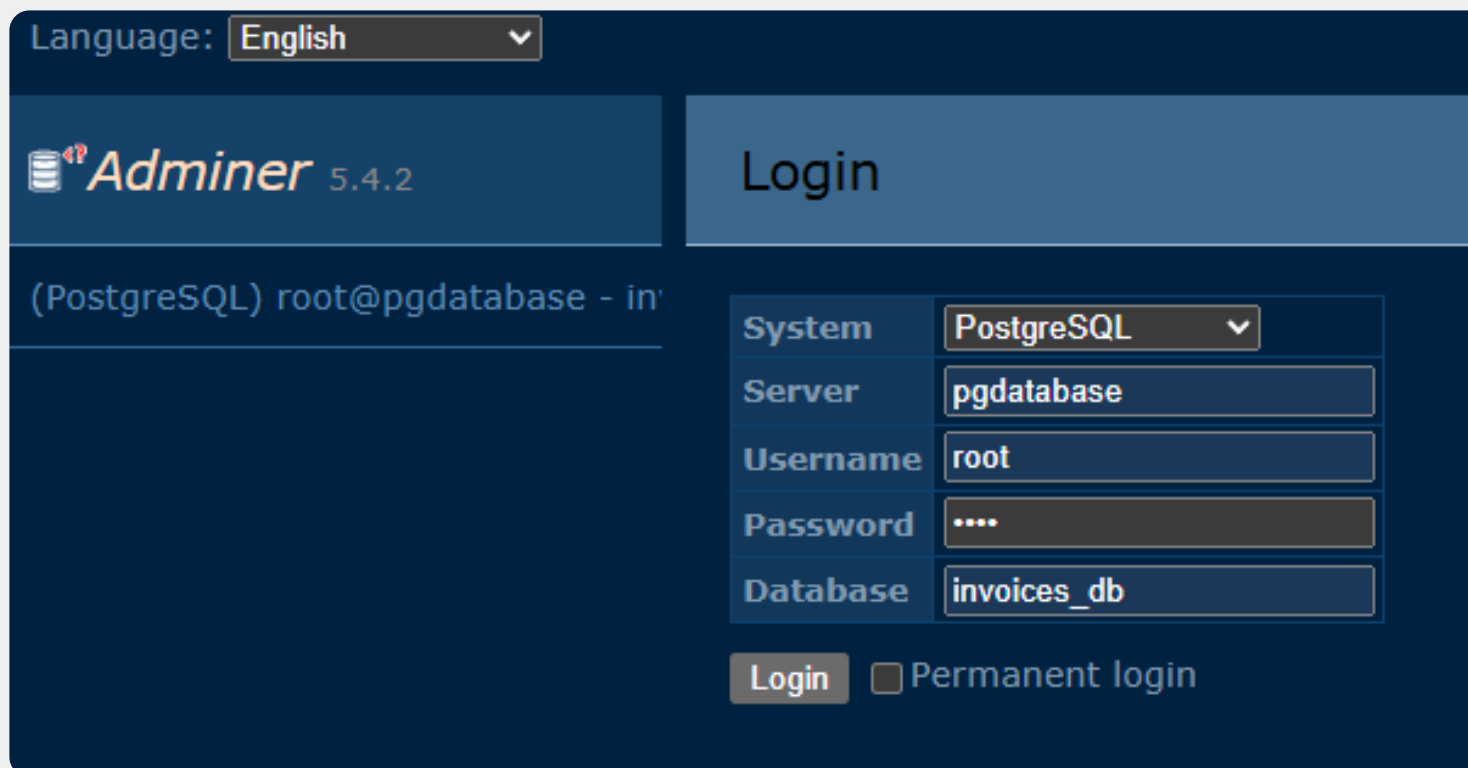
The result after the run is complete.

6. Check the result

Open the **Adminer** page

Go to

<http://localhost:8080/>



Language: **English** ▼

Adminer 5.4.2

(PostgreSQL) root@pgdatabase - in

Login

System	PostgreSQL ▼
Server	pgdatabase
Username	root
Password	
Database	invoices_db

Login ☐ Permanent login

Login to the Adminer

System: **PostgreSQL**
Server: **pgdatabase**
Username: **root**
Password: **root**
Database: **invoices_db**

6. Check the result

Language: English PostgreSQL » pgdatabase » invoices_db » Schema: public

Adminer 5.4.2

DB: invoices_db Schema: public

SQL command Import
Export Create table

select country
select customer
select invoice
select invoice_line_item
select payment
select product
select units

Schema: public

Alter schema Database schema Routines Sequences User types

Tables and views

Search data in tables (7)

LIKE %% Search

☐	Table	Engine	Data Length?	Index Length?	Data Free	Auto Increment	Rows?	Comment?
☐	country	table	16,384	32,768	?	?	?	
☐	customer	table	16,384	32,768	?	?	?	
☐	invoice	table	16,384	49,152	?	?	~ 51	
☐	invoice_line_item	table	40,960	65,536	?	?	~ 152	
☐	payment	table	16,384	16,384	?	?	?	
☐	product	table	16,384	32,768	?	?	?	
☐	units	table	16,384	32,768	?	?	?	
	7 in total		en_US.utf8	139,264	262,144	0		

Selected (0)

Vacuum Optimize Truncate Drop

Move to other database (0)

public Move

The payment table is now displayed.

6. Check the result

Language: English PostgreSQL » pgdatabase » invoices_db » public » Table: payment

Adminer 5.4.2

DB: invoices_db Schema: public

SQL command Import Export Create table

select country
select customer
select invoice
select invoice_line_item
select **payment**
select product
select units

Table: payment

Select data **Show structure** Alter table New item

Column	Type	Comment
id	bigint <i>Auto Increment</i> [GENERATED BY DEFAULT AS IDENTITY]	
invoice_id	bigint	
payment_date	date	
amount	numeric(12,2)	
method	text <i>NULL</i>	
note	text <i>NULL</i>	

Indexes

PRIMARY id

Alter indexes

Foreign keys

Source	Target	ON DELETE	ON UPDATE	
invoice_id	invoice(id)	CASCADE	NO ACTION	Alter

Add foreign key

Checks

Create check

Triggers

Add trigger

You can click on the table name to view the table structure.

6. Check the result

Language: English PostgreSQL » pgdatabase » invoices_db » public » Select: payment

Adminer 5.4.2

DB: invoices_db
Schema: public

SQL command Import
Export Create table

select country
select customer
select invoice
select invoice_line_item
select payment
select product
select units

Select data Show structure Alter table New item

Select Search Sort Limit Text length Action

50 100 Select

`SELECT * FROM "payment" LIMIT 50` (0.001 s) Edit

☐ Modify	id	invoice_id	payment_date	amount	method	note
☐ edit	1	2	2026-03-01	1500.00	bank_transfer	NULL
☐ edit	2	3	2026-03-02	3200.00	cash	NULL

Whole result Modify Selected (0) Export (2)

☐ 2 rows Save Edit Clone Delete

Import

You can click on "select" next to the table name to view the data in that table.

6. Check the result

```
339 INSERT INTO invoice_line_item (id, created_at, invoice_id, product_id, quantity, unit_price, extended_price) VALUES (148, '2026-02-02 10:33:00+00:00', 51, 20, 2.0, 30.75, 61.50);
340 INSERT INTO invoice_line_item (id, created_at, invoice_id, product_id, quantity, unit_price, extended_price) VALUES (149, '2026-02-02 10:34:00+00:00', 52, 8, 12.0, 60.00, 720.00);
341 INSERT INTO invoice_line_item (id, created_at, invoice_id, product_id, quantity, unit_price, extended_price) VALUES (150, '2026-02-02 10:35:00+00:00', 52, 17, 20.0, 90.50, 1810.0);
342 INSERT INTO invoice_line_item (id, created_at, invoice_id, product_id, quantity, unit_price, extended_price) VALUES (151, '2026-02-02 10:36:00+00:00', 52, 9, 20.0, 250.25, 5005.0);
343 INSERT INTO invoice_line_item (id, created_at, invoice_id, product_id, quantity, unit_price, extended_price) VALUES (152, '2026-02-02 10:37:00+00:00', 52, 20, 7.0, 30.75, 215.25);
344
345 INSERT INTO payment (id, invoice_id, payment_date, amount, method, note) VALUES (1, 2, '2026-03-01 00:00:00+00:00', 1500.00, 'bank_transfer', NULL) ON CONFLICT DO NOTHING;
346 INSERT INTO payment (id, invoice_id, payment_date, amount, method, note) VALUES (2, 3, '2026-03-02 01:00:00+00:00', 3200.00, 'cash', NULL) ON CONFLICT DO NOTHING;
347
348 -- update sequences to max id (crutial when inserting explicit IDs)
349 SELECT setval(pg_get_serial_sequence('country', 'id'), coalesce(max(id),0) + 1, false) FROM country;
350 SELECT setval(pg_get_serial_sequence('units', 'id'), coalesce(max(id),0) + 1, false) FROM units;
351 SELECT setval(pg_get_serial_sequence('customer', 'id'), coalesce(max(id),0) + 1, false) FROM customer;
```

Language: English PostgreSQL » pgdatabase » invoices_db » public » Select: payment

Adminer 5.4.2

DB: invoices_db Schema: public

SQL command Import
Export Create table

select country
select customer
select invoice
select invoice_line_item
select payment
select product
select units

Select data Show structure Alter table New item

Select Search Sort Limit 50 Text length 100 Action Select

SELECT * FROM "payment" LIMIT 50 (0.001 s) Edit

☐ Modify	id	invoice_id	payment_date	amount	method	note
☐ edit	1	2	2026-03-01	1500.00	bank_transfer	NULL
☐ edit	2	3	2026-03-02	3200.00	cash	NULL

Whole result ☐ 2 rows Modify Selected (0) Export (2)

Import

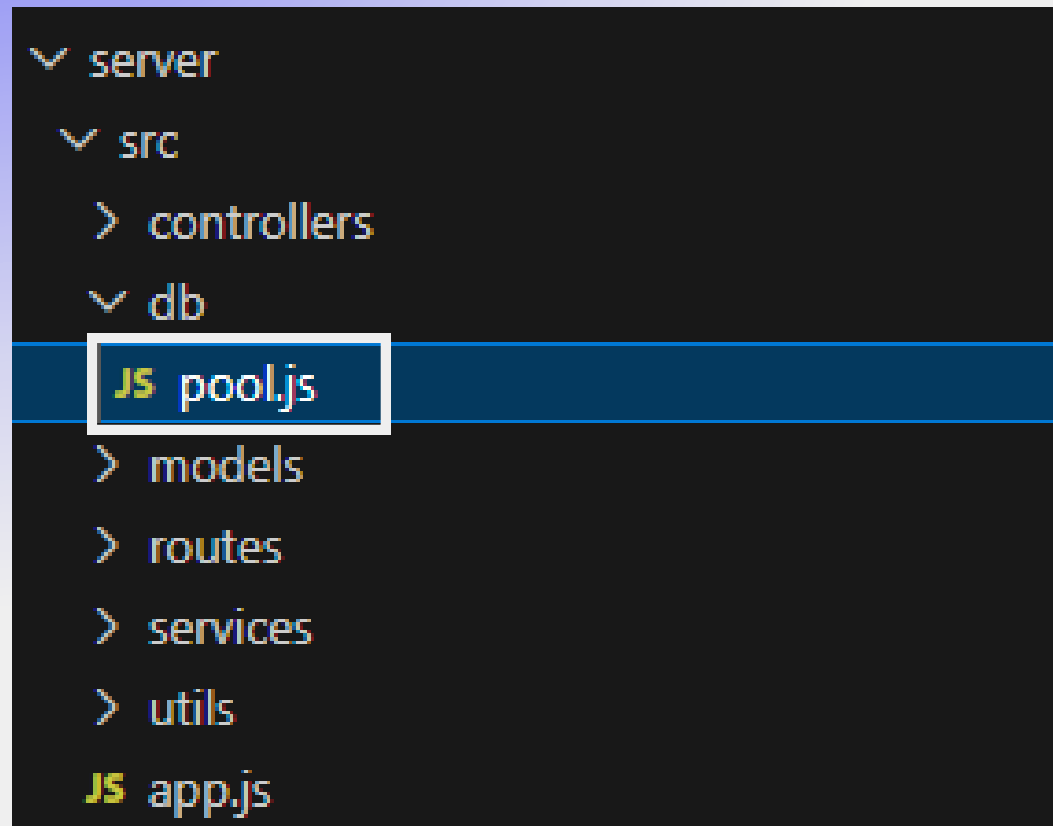
You should see the data from your seed in this table.

Extra

How does the backend connect to the database?

Connect backend to database

File: server/src/db/pool.js

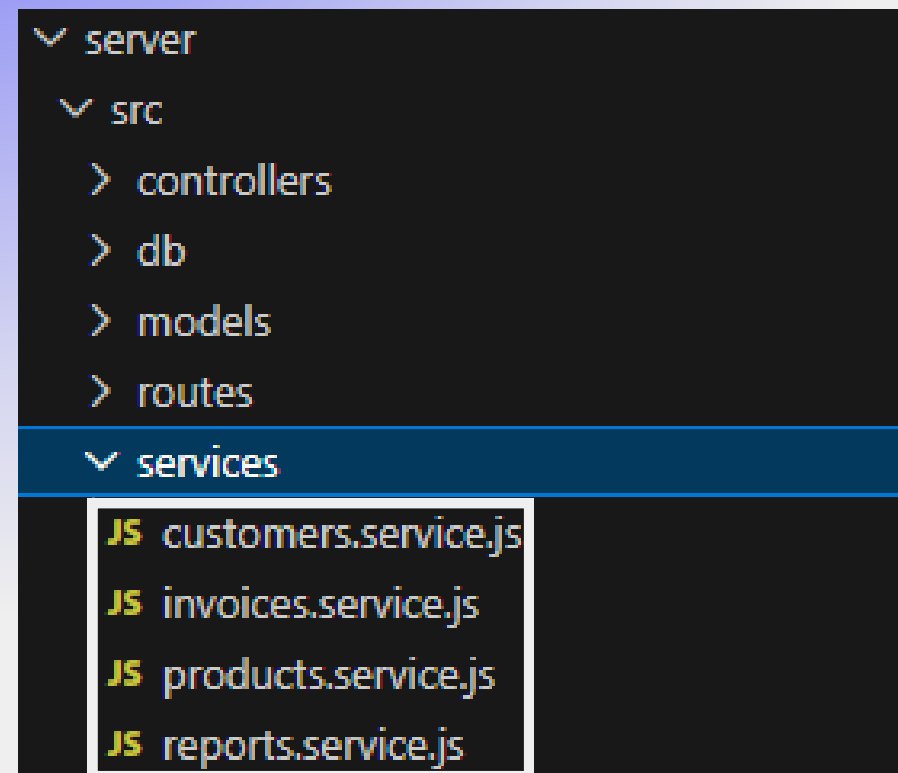


```
JS pool.js X
server > src > db > JS pool.js > ...
1  // PostgreSQL connection pool
2  // Example usage: pool.query("SELECT 1")
3  import pg from "pg";
4  import dotenv from "dotenv";
5  import logger from "../utils/logger.js";
6
7  dotenv.config();
8
9  // Production requires DATABASE_URL; dev may use default for convenience.
10 const connectionString =
11   process.env.DATABASE_URL ||
12   (process.env.NODE_ENV === "production"
13     ? (() => {
14         throw new Error("DATABASE_URL is required in production. Set it in your environment.");
15       })()
16     : "postgresql://root:root@localhost:15432/invoices_db");
17
18 export const pool = new pg.Pool({ connectionString });
19
20 pool.on("error", (err) => {
21   logger.error("Database pool error", { message: err.message });
22 });
23
```

The backend will connect to PostgreSQL via the [pool.js](#) file.

Connect backend to database

File: server/src/services/



Example services

See more in backend part!

```
JS customers.service.js X
server > src > services > JS customers.service.js > ...
1 // Customer CRUD and list (search by code/name/address, pagination, sort). Parameterized queries only.
2 import { pool } from "../db/pool.js";
3
4 export async function listCustomers({
5   search = "",
6   page = 1,
7   limit = 10,
8   sortBy = "name",
9   sortDir = "asc",
10 } = {}) {
11   const offset = (Number(page) - 1) * Number(limit);
12
13   const allowedSort = ["code", "name", "address_line1", "credit_limit"];
14   const sortColumn = allowedSort.includes(sortBy) ? sortBy : "name";
15   const sortDirection = sortDir === "asc" ? "ASC" : "DESC";
16
17   const searchParam = `%${search}%`;
18
19   const countResult = await pool.query(
20     `
21     SELECT COUNT(*) as total FROM customer
22     WHERE code ILIKE $1 OR name ILIKE $1 OR address_line1 ILIKE $1
23     `,
24     [searchParam],
25   );
26   const total = Number(countResult.rows[0].total);
27
28   const { rows } = await pool.query(
29     `
30     SELECT code, name, address_line1, address_line2, country_id, credit_limit, created_at
31     FROM customer
32     WHERE code ILIKE $1 OR name ILIKE $1 OR address_line1 ILIKE $1
33     ORDER BY ${sortColumn} ${sortDirection} NULLS LAST
34     LIMIT $2 OFFSET $3
35     `,
36     [searchParam, Number(limit), offset],
37   );
38 }
```

Import pool for database connection

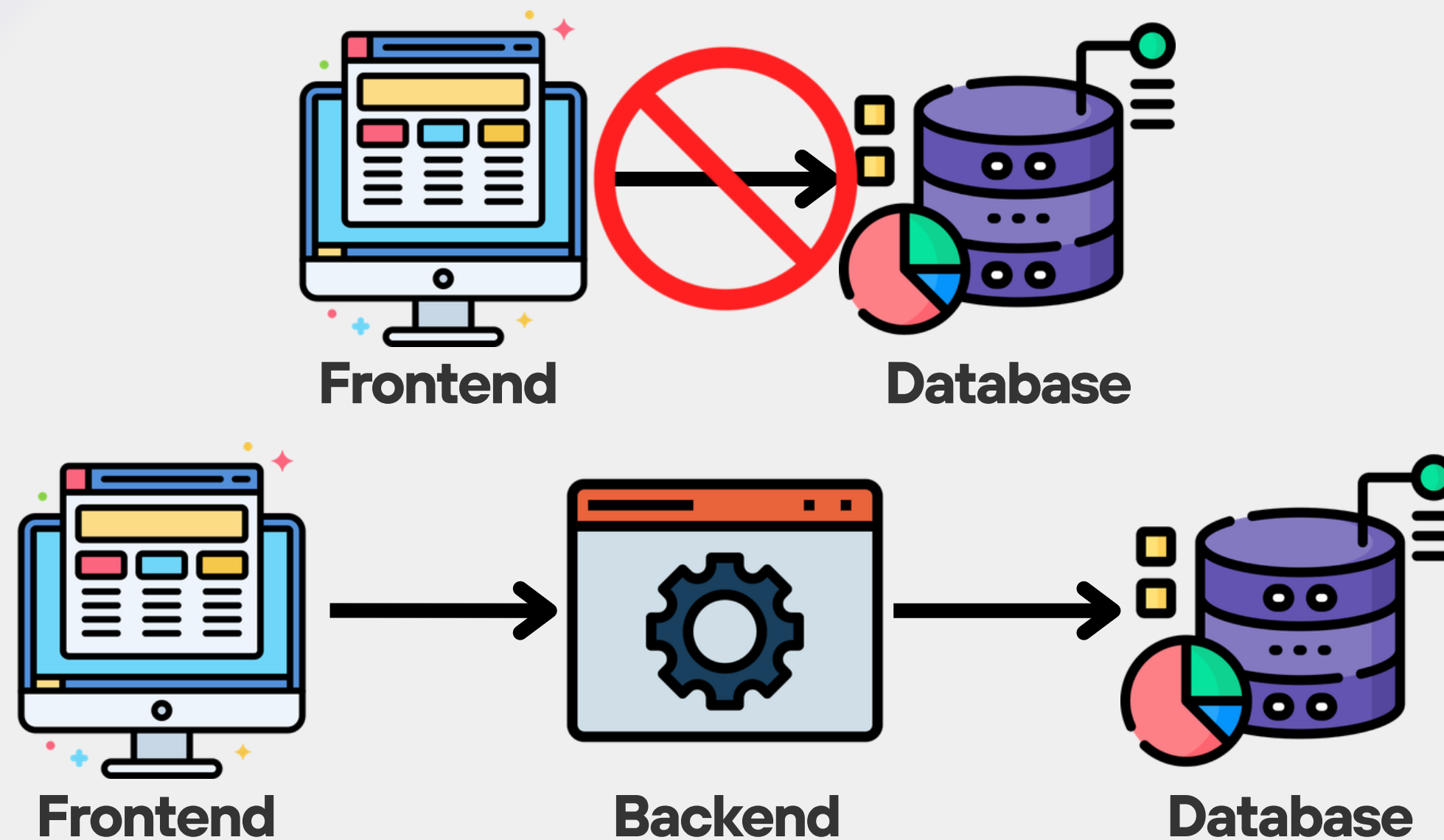
Use pool for database query

Part 4

Backend

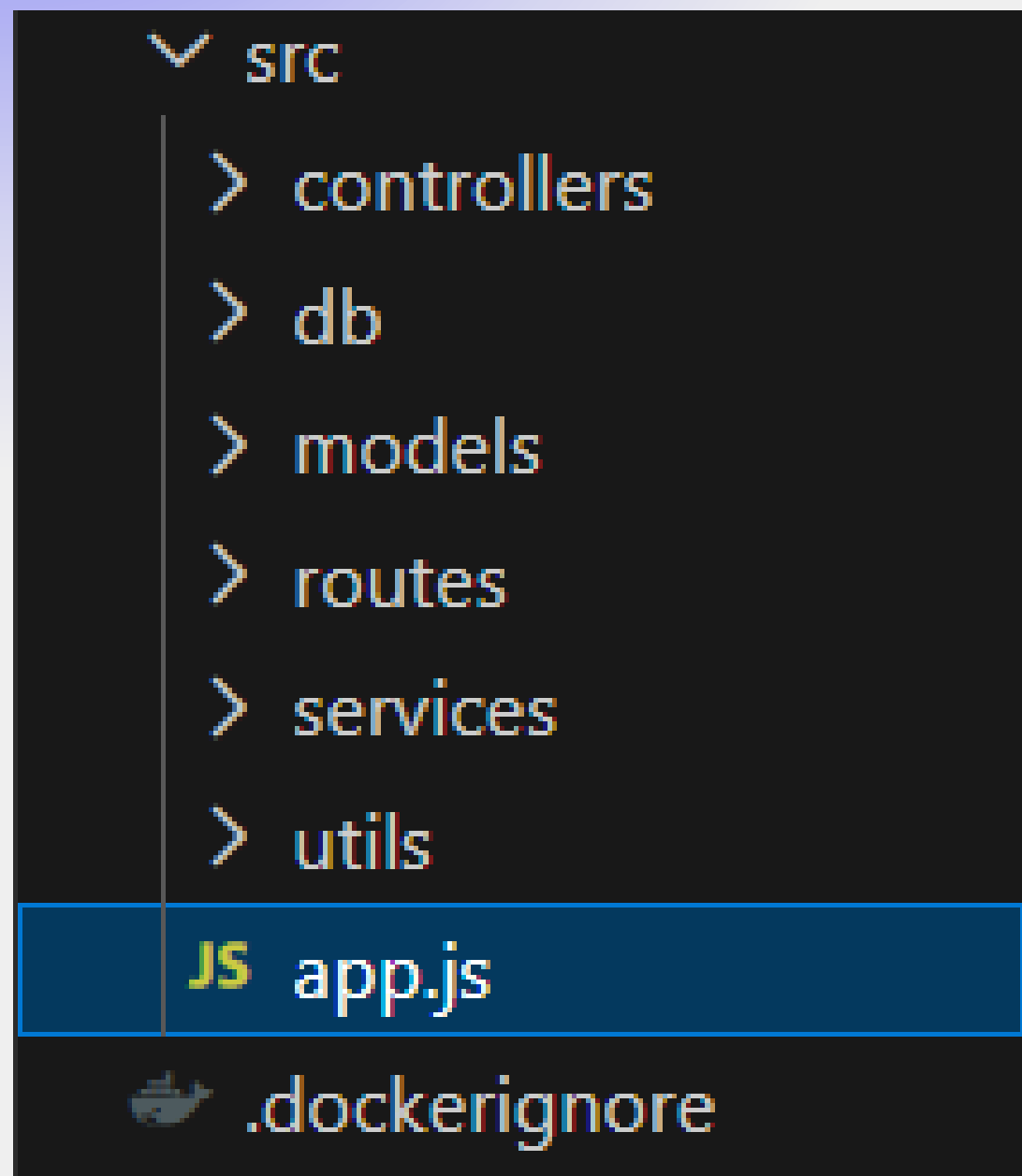
Why we need backend?

- **Frontend** will not directly connect with database, but it will **send request** to the **Backend** first.
- The Backend of this project work as **medium** between Frontend and Database.



Backend Entry Point

Path: server/src/app.js



- This file is considered as the main entry point of the program.
- Its responsibilities are:
 - Creating the Express app
 - Configuring CORS*
 - Enabling JSON request handling
 - Performing request logging
 - And connecting various routes to the system
- Think of it as main() in C programming

*CORS (Cross-Origin Resource Sharing) is a browser security mechanism that allows servers to control which websites are permitted to access their resources.

Route Registration in app.js

Path: server/src/app.js

```
// API routes
app.use("/api/customers", customersRoutes);
app.use("/api/products", productsRoutes);
app.use("/api/invoices", invoicesRoutes);
app.use("/api/reports", reportsRoutes)
```

Based from these lines, it's mean that:

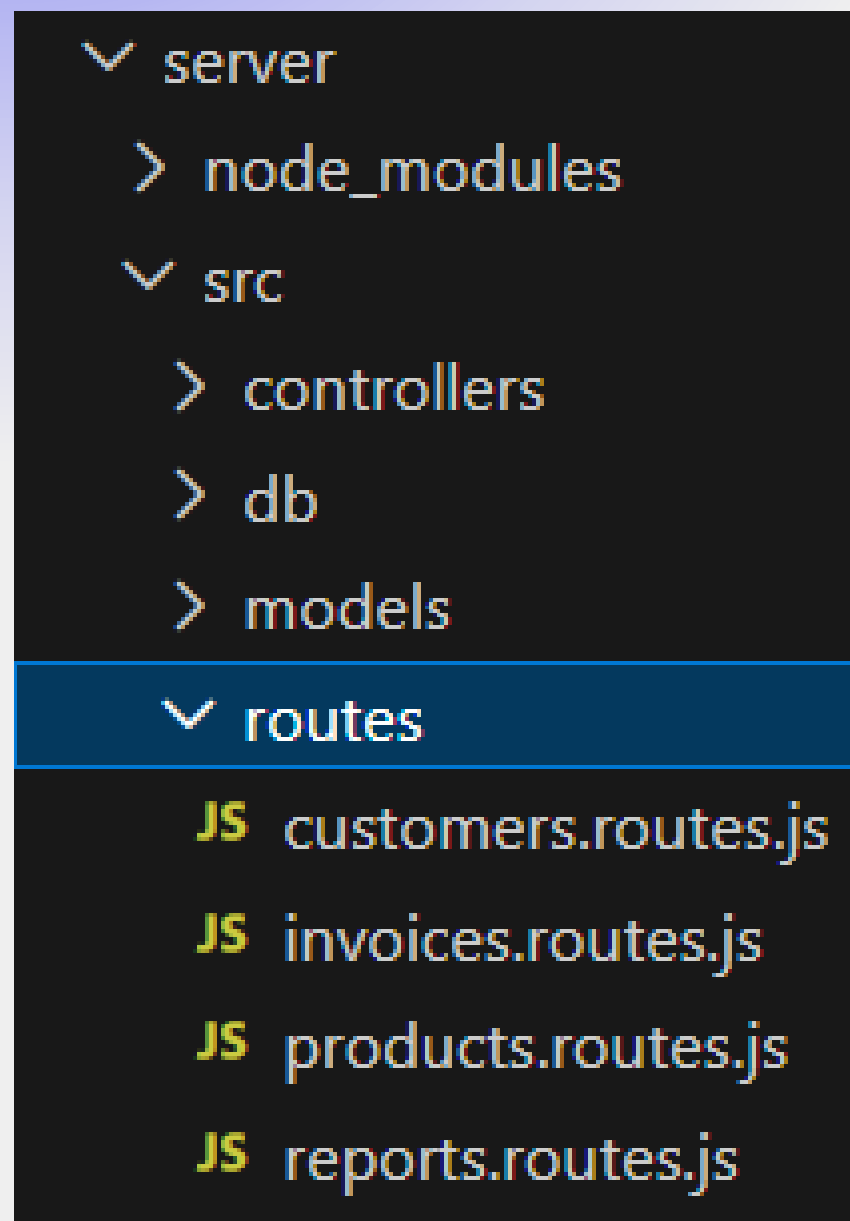
- If a request comes to /api/customers, it will be forwarded to the customers route
- If a request comes to /api/products, it will go to the products route

Therefore, app.js does not contain business logic itself.

It only acts as a central place that receives and distributes requests to different modules.

Routes Layer

Path: server/src/routes/



- This layer is responsible for **mapping URLs to the functions that should handle them.**
- In this project, the main route files include:
 - "customers.routes.js"
 - "products.routes.js"
 - "invoices.routes.js"
 - "reports.routes.js"

Route Example

Path: server/src/routes/customers.routes.js

✓ server	
> node_modules	
✓ src	
> controllers	
> db	
> models	
✓ routes	
	JS customers.routes.js
	JS invoices.routes.js
	JS products.routes.js
	JS reports.routes.js

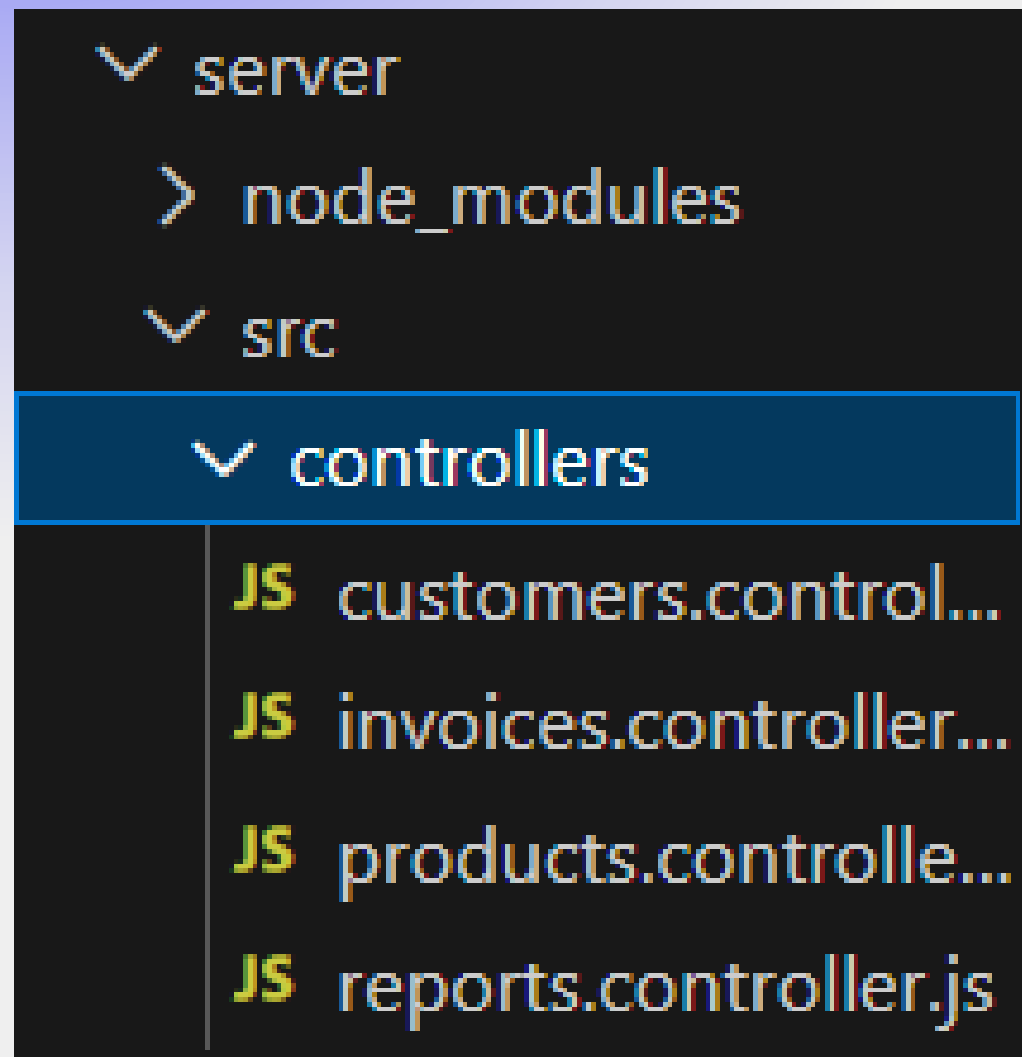
In customers.routes.js, we will see a pattern like this:

```
// List customers with pagination, search, sort
r.get("/", c.listCustomers);
// Create customer (supports auto code)
r.post("/", c.createCustomer);
r.get("/countries", c.listCountries);
// Get/update/delete by business key (code), not primary key
r.put("/:code", c.updateCustomer);
r.delete("/:code", c.deleteCustomer);
r.get("/:code", c.getCustomer);
```

- The route layer has only one main responsibility:
 - mapping a request URL to the controller function that should handle it.
- The route does **not interact with the database directly**.
- It simply acts as a **request dispatcher**.

Controllers Layer

Path: server/src/controllers/



- After the route receives the request, it forwards it to a controller.
- Controllers are located in the server/src/controllers folder.
- The responsibilities of a controller include:
 - Reading req.params
 - Reading req.query
 - Reading req.body
 - Validating input data
 - Calling the service layer
 - Sending the response back

Controller Example

Path: server/src/controllers/customers.controller.js

✓	server
>	node_modules
✓	src
✓	controllers
	JS customers.control...
	JS invoices.controller...
	JS products.controlle...
	JS reports.controller.js

- If we open customers.controller.js, we will see that the controller does not query the database directly.
- Instead, it calls customersService.

```
const result = await customersService.listCustomers(req.query);
sendList(res, result);
```

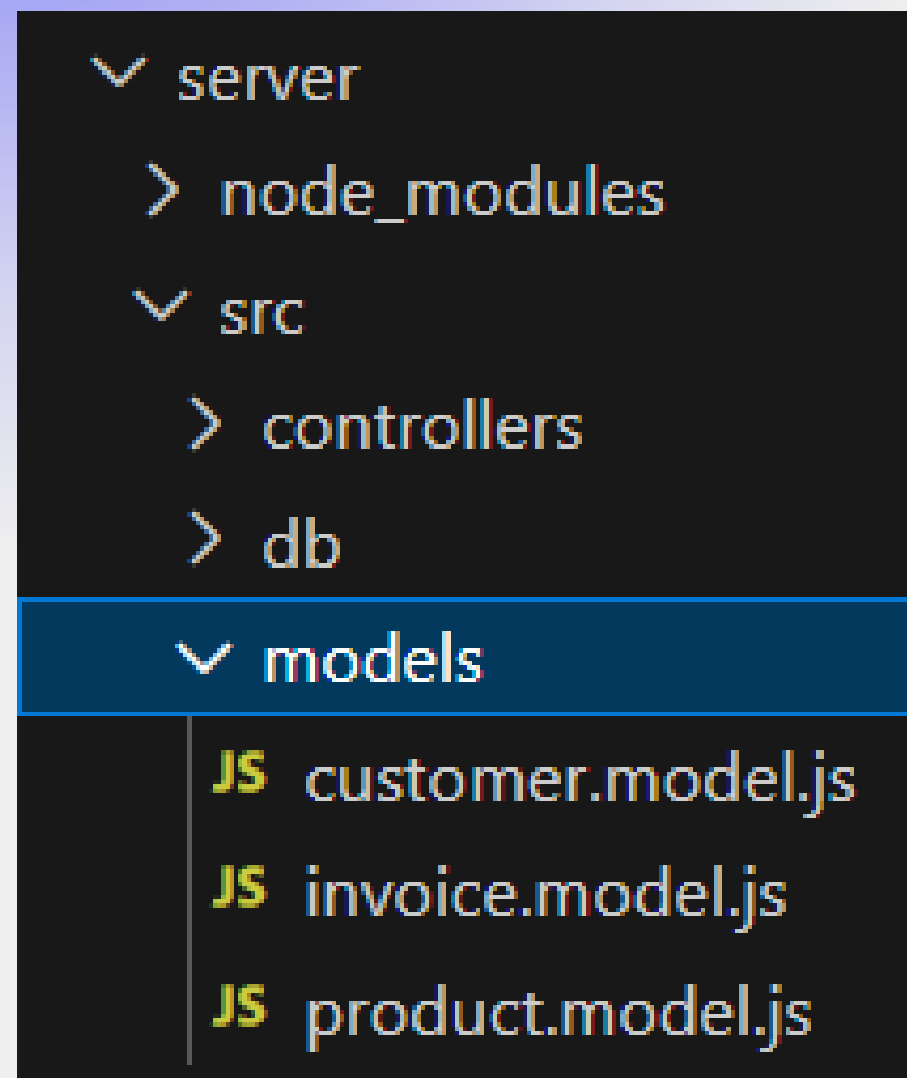
- Or when creating a customer:

```
const parsed = CreateCustomerBodySchema.safeParse(req.body);
const result = await customersService.createCustomer(parsed.data);
sendCreated(res, result);
```

- This clearly shows the flow:
 - Controller acts as the middle layer between routes and services.

Models Validation

Path: server/src/models



- Another good practice in this project is that there are models separated in the server/src/models folder.
- The project uses **Zod*** to validate the request body before sending it to the service layer.
- For example:

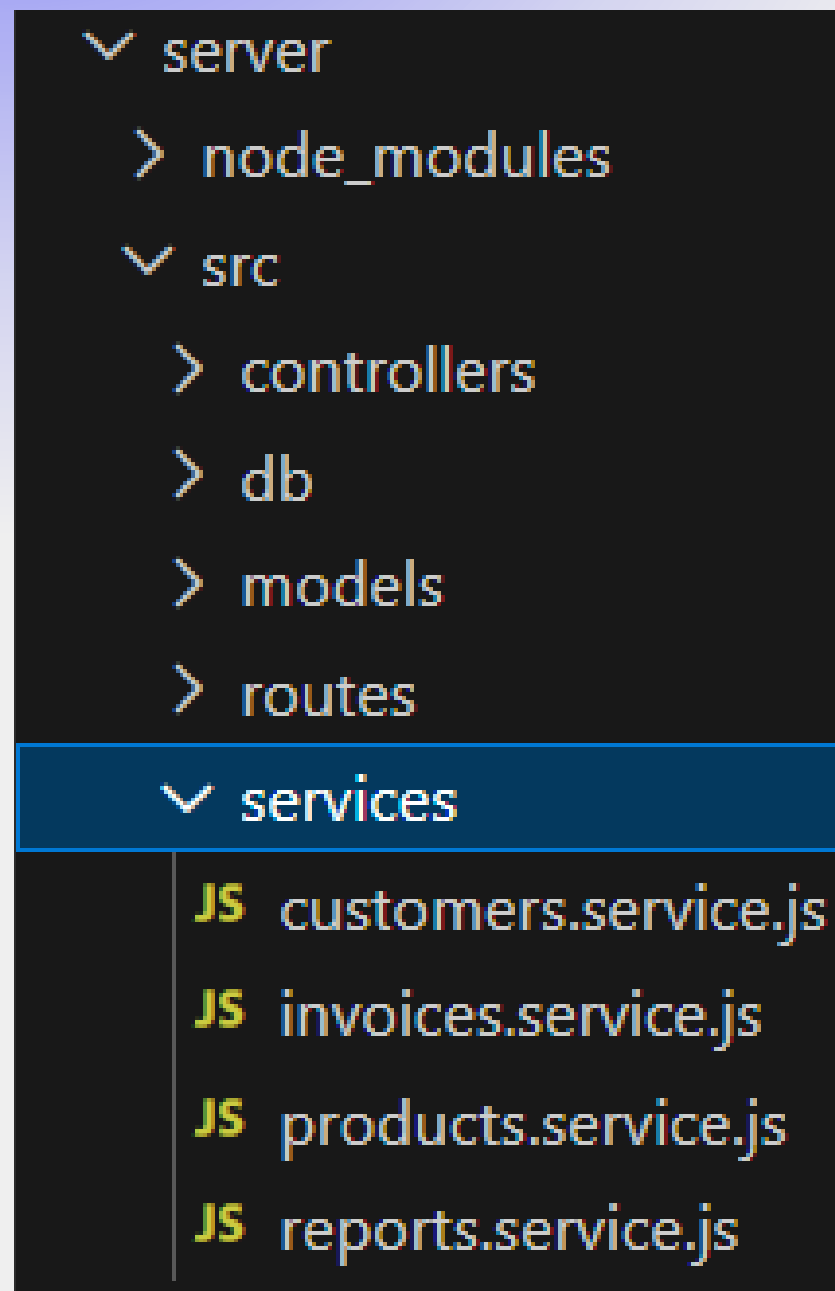
```
export const CreateCustomerBodySchema = z.object({  
  // ...  
})  
  
export const CreateInvoiceSchema = z.object({  
  // ...  
})
```

- If the request data from the frontend is incomplete or has the wrong data type, the controller will immediately return an error without touching the database.
- This helps improve backend security and data integrity.

*Zod is a TypeScript-first validation library. Using Zod, you can define schemas you can use to validate data, from a simple string to a complex nested object.

Services Layer

Path: server/src/services/



- Next is the most important layer of the backend — the **service layer**.
- Services are located in server/src/services.
- This is where the business logic and database queries are implemented.
- Example files include:
 - customers.service.js
 - products.service.js
 - invoices.service.js
 - reports.service.js

Customer Service Example

Path: server/src/services/customers.service.js

```
export async function listCustomers({
  search = "",
  page = 1,
  limit = 10,
  sortBy = "name",
  sortDir = "asc",
} = {}) {
```

- If we open customers.service.js, we will see actual SQL statements being used: SELECT, INSERT, UPDATE, DELETE
- For example, listCustomers receives query parameters such as: search, page, limit, sortBy, sortDir
- Then it constructs a SQL query to retrieve data from the customer table.
- This is essentially the core of the backend, where requests are translated into database operations.

Invoice Service Example

Path: server/src/services/invoices.service.js

```
const client = await pool.connect();
try {
  await client.query("begin");

  const code = customer_code !== null ? String(customer_code).trim() : "";
  const cust = await client.query("SELECT id, credit_limit FROM customer WHERE code = $1", [code]);
  if (cust.rowCount === 0) throw new Error(`Customer not found: ${code}`)
  const customer_id = cust.rows[0].id;
```

```
const inv = await client.query(
  insert into invoice (id, created_at, invoice_no, invoice_date,
customer_id, total_amount, vat, amount_due)
  values (
    (select coalesce(max(id),0)+1 from invoice),
    now(),
    $1,$2,$3,$4,$5,$6
  )
  ,
  returning id, invoice_no
  ,
  [resolvedInvoiceNo, invoice_date, customer_id, total, vat, amount_due],
);
```

```
await client.query("COMMIT");
```

```
await client.query("ROLLBACK");
```

- It is quite complex because an invoice is not just a single header record. It also contains line items.
- So when creating or updating an invoice, the service has to work with multiple records together.
- This file also uses transactions, which means all steps must succeed together, or the changes are rolled back if an error happens.
- This is a very good example of backend business logic in a real project.

Database Connection

Path: server/src/db/pool.js

```
// Production requires DATABASE_URL; dev may use default for convenience.
const connectionString =
  process.env.DATABASE_URL ||
  (process.env.NODE_ENV === "production"
    ? (() => {
        throw new Error("DATABASE_URL is required in production. Set it in your environment.");
      })()
    : "postgresql://root:root@localhost:15432/invoices_db");

export const pool = new pg.Pool({ connectionString });
```

- how do services connect to the database?
- That connection is defined in server/src/db/pool.js.
- This file creates a PostgreSQL connection pool and exports it for the service layer to use.
- In short, services use pool.query() to send SQL to PostgreSQL.

Response Helper

Path: server/src/db/pool.js

```
export function sendList(res, payload, status = 200) {
  const { data, total, page, limit, totalPages } = payload;
  success(res, data, status, { meta: { total, page, limit, totalPages } });
}

export function sendData(res, data, status = 200, message = null) {
  success(res, data, status, message ? { message } : {});
}

export function sendOne(res, data, status = 200) {
  success(res, data, status);
}

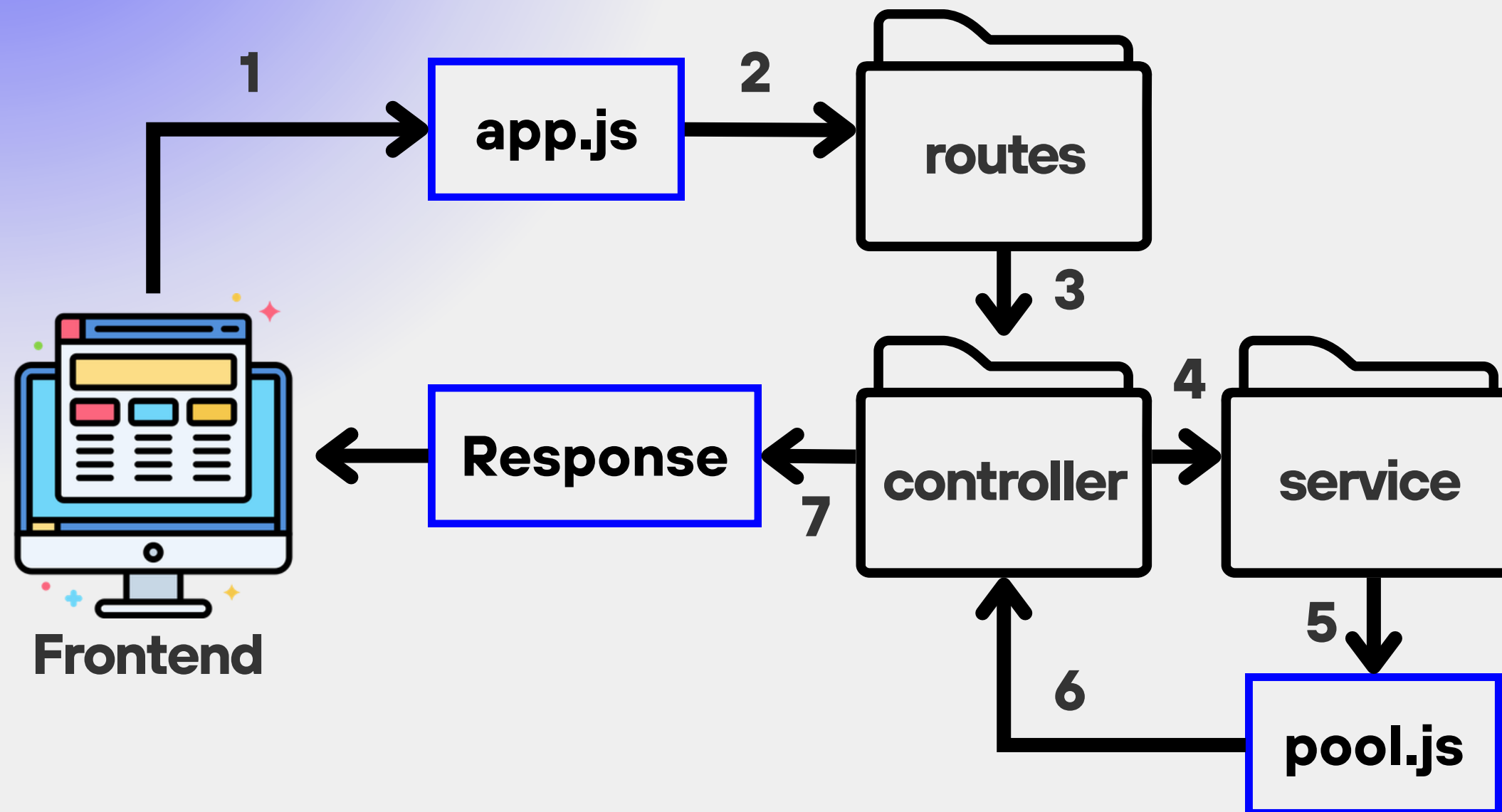
export function sendCreated(res, data, status = 201) {
  success(res, data, status);
}

export function sendOk(res, payload = { ok: true }, status = 200) {
  success(res, payload, status);
}

export function sendError(res, message, status = 500, code = null, details = null) {
  if (!code) {
    if (status === 404) code = "NOT_FOUND";
    else if (status === 400) code = "BAD_REQUEST";
    else if (status === 422) code = "VALIDATION_ERROR";
    else code = "INTERNAL_ERROR";
  }
  fail(res, message, status, code, details);
}
```

- After the service layer finishes its work, the controller sends the response back.
- This project uses helper functions in server/src/utils/response.js included:
 - sendList
 - sendOne
 - sendCreated
 - sendOk
 - sendError
- The benefit is that API responses follow a consistent structure across the whole backend.
- That makes the frontend easier to build and maintain.

Full Backend Flow



1. The frontend sends a request.
2. app.js receives it and forwards it to the correct route.
3. The route selects the matching controller.
4. The controller reads request data and calls the service.
5. The service queries the database through pool.js.
6. The result comes back to the controller.
7. The controller sends the final response using the response helper.

Adding new endpoint / module

Step 1: adding new route

Path: server/src/routes/payments.routes.js (create)

```
server\src\routes\payments.routes.js
@@ -0,0 +1,13 @@
✓ ✓ 1 + // Payment API routes
✓ 2 + // Example usage: GET /api/payments
✓ 3 + import { Router } from "express";
✓ 4 + import * as c from "../controllers/payments.controller.js";
✓ 5 +
✓ 6 + const r = Router();
✓ 7 +
✓ 8 + // List payments with pagination
✓ 9 + r.get("/", c.listPayments);
✓ 10 + r.post("/", c.createPayment);
✓ 11 + r.get("/:id", c.getPayment);
✓ 12 +
✓ 13 + export default r;
```

- If you want to add a new feature to the backend, the first thing you need is a new route.
- The route maps an endpoint to a controller.
- For example, if we want to build a payment module, we might have GET /api/payments, POST /api/payments, and GET /api/payments/:id.

Step 2: adding new controller

Path: server/src/routes/payments.controller.js (create)

```
server\src\controllers\payments.controller.js
@@ -0,0 +1,45 @@
✓ ✓ 1 + import * as paymentsService from "../services/payments.service.js";
✓ 2 + import { CreatePaymentBodySchema } from "../models/payment.model.js";
✓ 3 + import { sendList, sendCreated, sendOne, sendError } from "../utils/response.js";
✓ 4 + import logger from "../utils/logger.js";
✓ 5 +
✓ 6 + export async function listPayments(req, res) {
✓ 7 +   try {
✓ 8 +     const result = await paymentsService.listPayments(req.query);
✓ 9 +     sendList(res, result);
✓ 10 +   } catch (err) {
✓ 11 +     logger.error("listPayments failed", { error: err?.message ?? String(err) });
✓ 12 +     sendError(res, err?.message ?? String(err), 500);
✓ 13 +   }
✓ 14 + }
✓ 15 +
✓ 16 + export async function createPayment(req, res) {
✓ 17 +   const parsed = CreatePaymentBodySchema.safeParse(req.body);
✓ 18 +   if (!parsed.success) {
✓ 19 +     return sendError(res, "Validation failed", 400, "VALIDATION_ERROR", parsed.error.flatten());
✓ 20 +   }
✓ 21 +
✓ 22 +   try {
✓ 23 +     const result = await paymentsService.createPayment(parsed.data);
✓ 24 +     sendCreated(res, result);
✓ 25 +   } catch (err) {
✓ 26 +     logger.error("createPayment failed", { error: err?.message ?? String(err) });
✓ 27 +     sendError(res, err?.message ?? String(err), 400);
✓ 28 +   }
✓ 29 + }
✓ 30 +
✓ 31 + export async function getPayment(req, res) {
✓ 32 +   const id = Number(req.params.id);
✓ 33 +   if (!Number.isInteger(id) || id <= 0) {
✓ 34 +     return sendError(res, "Invalid payment id", 400);
✓ 35 +   }
✓ 36 +
✓ 37 +   try {
✓ 38 +     const result = await paymentsService.getPayment(id);
✓ 39 +     if (!result) return sendError(res, "Payment not found", 404);
✓ 40 +     sendOne(res, result);
✓ 41 +   } catch (err) {
✓ 42 +     logger.error("getPayment failed", { id: req.params.id, error: err?.message ?? String(err) });
✓ 43 +     sendError(res, err?.message ?? String(err), 500);
✓ 44 +   }
✓ 45 + }
```

- Next, the controller is the layer that receives the actual request and then calls the service.
- If the feature has a request body that needs validation, we can also add a model schema and parse the data before calling the service, just like in other modules.

Step 3: adding new service

Path: server/src/routes/payments.service.js (create)

```
server\src\services\payments.service.js
@@ -0,0 +1,47 @@
✓ ✓ 1 + // Payment CRUD and list. Parameterized queries only.
✓ 2 + import { pool } from "../db/pool.js";
✓ 3 +
✓ 4 + export async function listPayments({ page = 1, limit = 10 } = {}) {
✓ 5 +   const offset = (Number(page) - 1) * Number(limit);
✓ 6 +
✓ 7 +   const countResult = await pool.query("SELECT COUNT(*) AS total FROM payment");
✓ 8 +   const total = Number(countResult.rows[0].total);
✓ 9 +
✓ 10 +   const { rows } = await pool.query(
✓ 11 +     `SELECT id, invoice_id, payment_date, amount, method, note
✓ 12 +     FROM payment
✓ 13 +     ORDER BY id DESC
✓ 14 +     LIMIT $1 OFFSET $2`,
✓ 15 +     [Number(limit), offset],
✓ 16 +   );
✓ 17 +
✓ 18 +   return {
✓ 19 +     data: rows,
✓ 20 +     total,
✓ 21 +     page: Number(page),
✓ 22 +     limit: Number(limit),
✓ 23 +     totalPages: Math.ceil(total / Number(limit)),
✓ 24 +   };
✓ 25 + }
```

```
✓ 26 +
✓ 27 + export async function createPayment({ invoice_id, payment_date, amount, method, note }) {
✓ 28 +   const { rows } = await pool.query(
✓ 29 +     `INSERT INTO payment (invoice_id, payment_date, amount, method, note)
✓ 30 +     VALUES ($1, $2, $3, $4, $5)
✓ 31 +     RETURNING id`,
✓ 32 +     [invoice_id, payment_date, amount, method, note],
✓ 33 +   );
✓ 34 +
✓ 35 +   return rows[0];
✓ 36 + }
✓ 37 +
✓ 38 + export async function getPayment(id) {
✓ 39 +   const { rows } = await pool.query(
✓ 40 +     `SELECT id, invoice_id, payment_date, amount, method, note
✓ 41 +     FROM payment
✓ 42 +     WHERE id = $1`,
✓ 43 +     [id],
✓ 44 +   );
✓ 45 +
✓ 46 +   return rows[0] ?? null;
✓ 47 + }
```

- The service layer is where the database is actually queried.
- Here we import pool from server/src/db/pool.js and use pool.query() to send SQL to PostgreSQL.
- So if you want to add a new module, the service layer is one of the most important parts you need.

Step 4: Register new route in app.js

Path: server/src/app.js

server\src\app.js			
↑		@@ -7,6 +7,7 @@ import fs from "fs";	
	7	7	
	8	8	import logger from "../utils/logger.js";
	9	9	import invoicesRoutes from "../routes/invoices.routes.js";
✓		10	+ import paymentsRoutes from "../routes/payments.routes.js";
	10	11	import reportsRoutes from "../routes/reports.routes.js";
	11	12	import customersRoutes from "../routes/customers.routes.js";
	12	13	import productsRoutes from "../routes/products.routes.js";
	106	107	app.use("/api/customers", customersRoutes);
	107	108	app.use("/api/products", productsRoutes);
	108	109	app.use("/api/invoices", invoicesRoutes);
✓		110	+ app.use("/api/payments", paymentsRoutes);
	109	111	app.use("/api/reports", reportsRoutes);
	110	112	
	111	113	const port = process.env.PORT 4000;

- After adding the route, controller, and service, do not forget to register the new route in app.js.
- Otherwise, the new endpoint will not be active yet.

Step 5: add model validation

Path: server/src/models/payment.model.js

```
server\src\models\payment.model.js
@@ -0,0 +1,18 @@
✓ ✓ 1 + import { z } from "zod";
✓ 2 +
✓ 3 + export const PaymentSchema = z.object({
✓ 4 +   id: z.number().int(),
✓ 5 +   invoice_id: z.number().int(),
✓ 6 +   payment_date: z.string().min(8),
✓ 7 +   amount: z.number().positive(),
✓ 8 +   method: z.string().nullable().optional(),
✓ 9 +   note: z.string().nullable().optional(),
✓ 10 + });
✓ 11 +
✓ 12 + export const CreatePaymentBodySchema = z.object({
✓ 13 +   invoice_id: z.coerce.number().int(),
✓ 14 +   payment_date: z.string().min(8),
✓ 15 +   amount: z.coerce.number().positive(),
✓ 16 +   method: z.string().optional(),
✓ 17 +   note: z.string().optional(),
✓ 18 + });
```

- If the incoming data has a clear structure,
- such as a payment or an invoice, it is a good idea to add a model schema for validation.
- This allows the controller to validate the data before calling the service.

Congratulation! You have done it.



How to run Backend?

Server run commands

Path: server/package.json

```
cd server  
npm install  
npm run dev
```

- If you want to run the backend for this project,
- go into the server folder and use these commands:

```
npm install  
npm run dev
```

- *This project uses node --watch src/app.js, so when you edit server files, it automatically reloads during development.*

Part 4

Frontend

Important Files & Folder

File: client/src/main.jsx

- Main entry file of the frontend

```
function Sidebar() {  
  // NavLink injects isActive when the current URL matches this link  
  const getLinkClass = ({ isActive }) => isActive ? "nav-item active" : "nav-item";  
  const getSubLinkClass = ({ isActive }) => isActive ? "nav-subitem active" : "nav-subitem";  
  
  return (  

```

Sidebar

```
function Layout({ children }) {  
  return (  

```

System Layout

```
ReactDOM.createRoot(document.getElementById("root")).render(  
  <React.StrictMode>  
    <BrowserRouter>  
      <Routes>
```

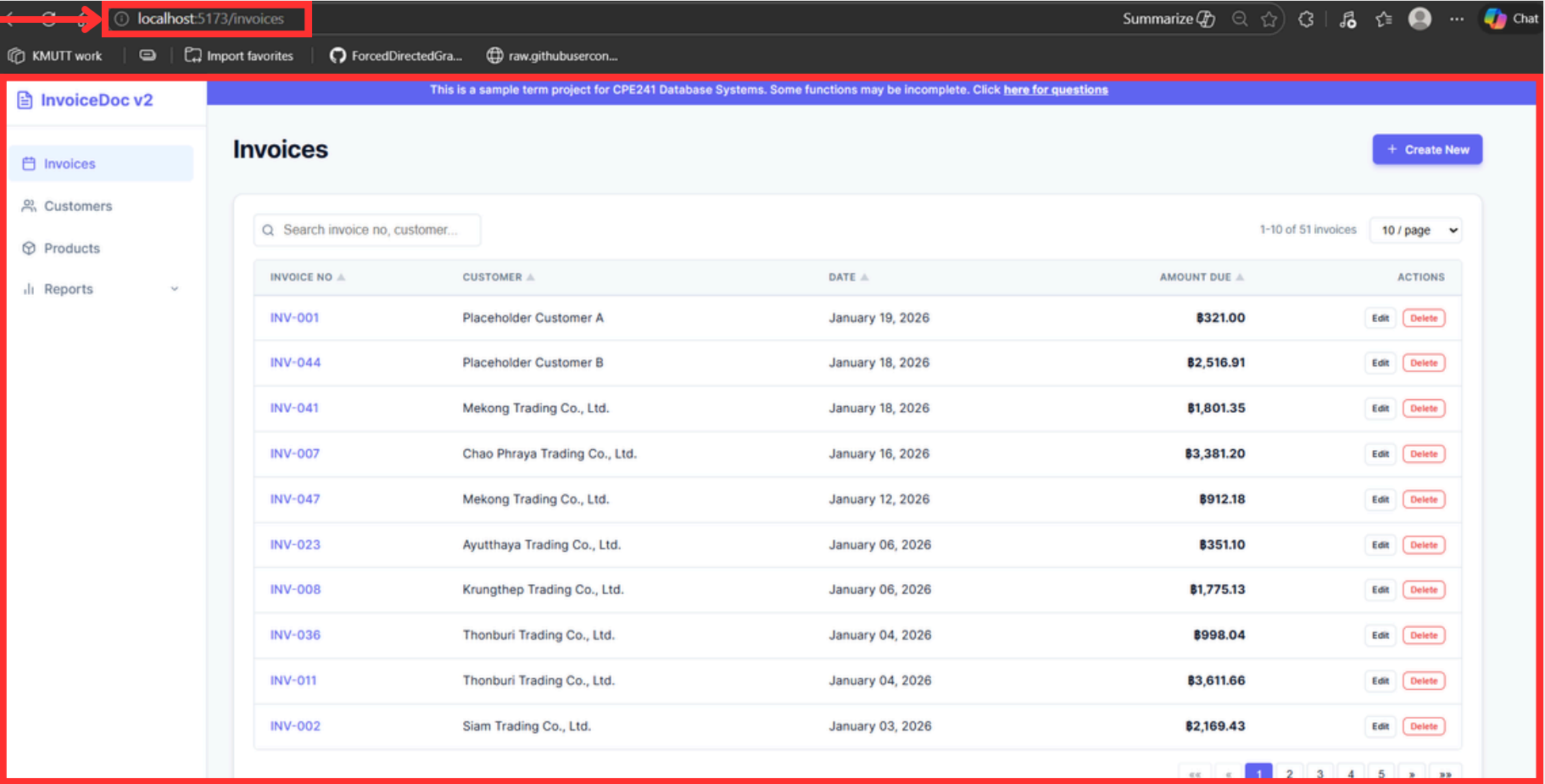
System Route

Routes

File: client/src/main.jsx

```
ReactDOM.createRoot(document.getElementById('root')).render(  
  <React.StrictMode>  
    <BrowserRouter>  
      <Routes>  
        {/* Default route → invoices list */}  
        <Route path="/" element={<Navigate to="/invoices" replace />} />  
        <Route path="/invoices" element={<Layout><InvoiceList /></Layout>} />  
        <Route path="/invoices/new" element={<Layout><InvoicePage mode="create" /></Layout>} />  
        <Route path="/invoices/:id" element={<Layout><InvoicePage mode="view" /></Layout>} />  
        <Route path="/invoices/:id/edit" element={<Layout><InvoicePage mode="edit" /></Layout>} />  
        <Route path="/customers" element={<Layout><CustomerList /></Layout>} />  
        <Route path="/customers/new" element={<Layout><CustomerPage mode="create" /></Layout>} />  
        <Route path="/customers/:id" element={<Layout><CustomerPage mode="view" /></Layout>} />  
        <Route path="/customers/:id/edit" element={<Layout><CustomerPage mode="edit" /></Layout>} />  
        <Route path="/products" element={<Layout><ProductList /></Layout>} />  
        <Route path="/products/new" element={<Layout><ProductPage mode="create" /></Layout>} />  
        <Route path="/products/:id" element={<Layout><ProductPage mode="view" /></Layout>} />  
        <Route path="/products/:id/edit" element={<Layout><ProductPage mode="edit" /></Layout>} />  
        <Route path="/reports" element={<Navigate to="/reports/product-sales" replace />} />  
        <Route path="/reports/product-sales" element={<Layout><Reports type="product-sales" /></Layout>} />  
        <Route path="/reports/monthly-sales" element={<Layout><Reports type="monthly-sales" /></Layout>} />  
        <Route path="/reports/customer-sales" element={<Layout><Reports type="customer-sales" /></Layout>} />  
      </Routes>  
    </BrowserRouter>  
  </React.StrictMode>  
</pre>
```

Code

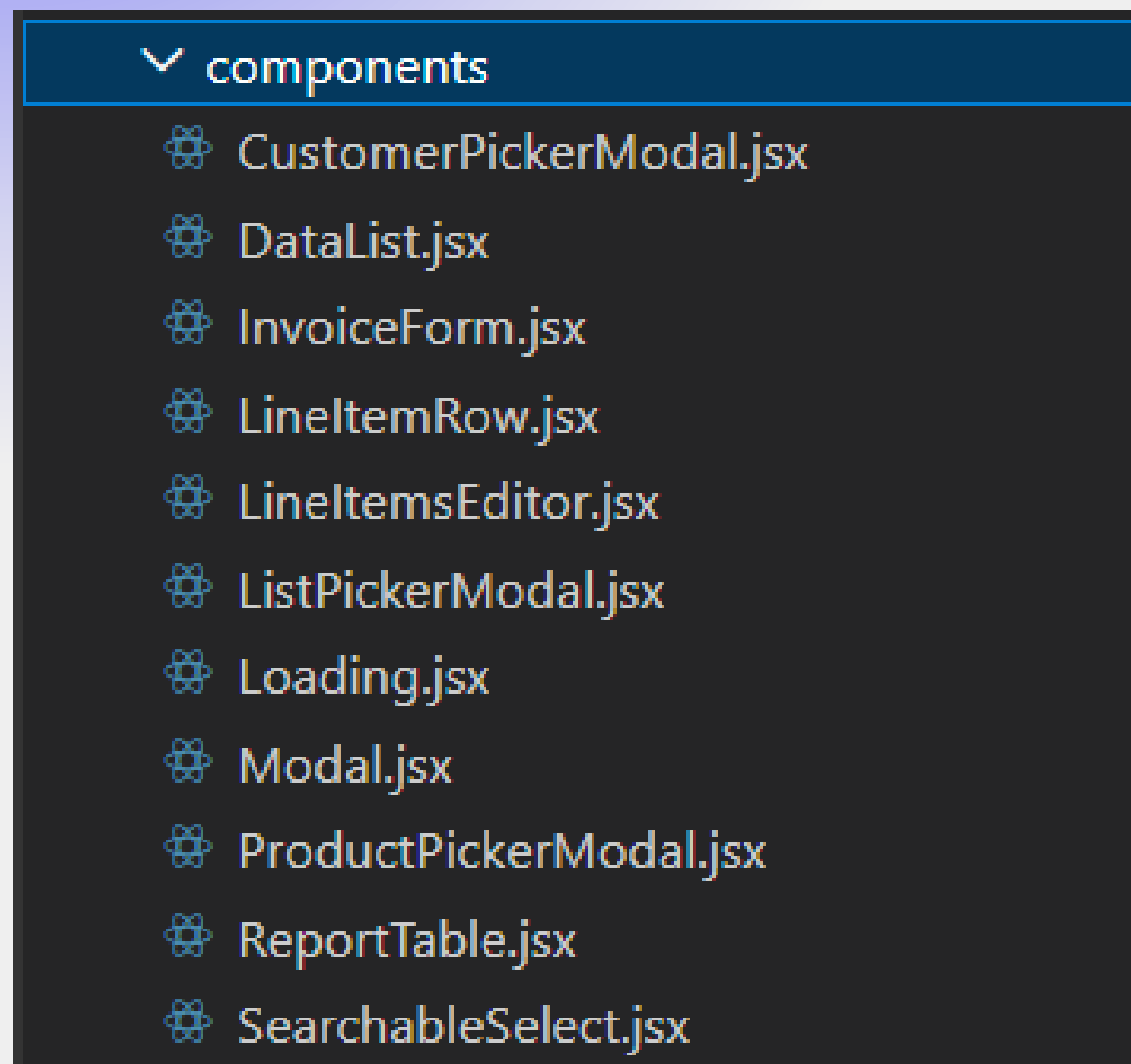


System

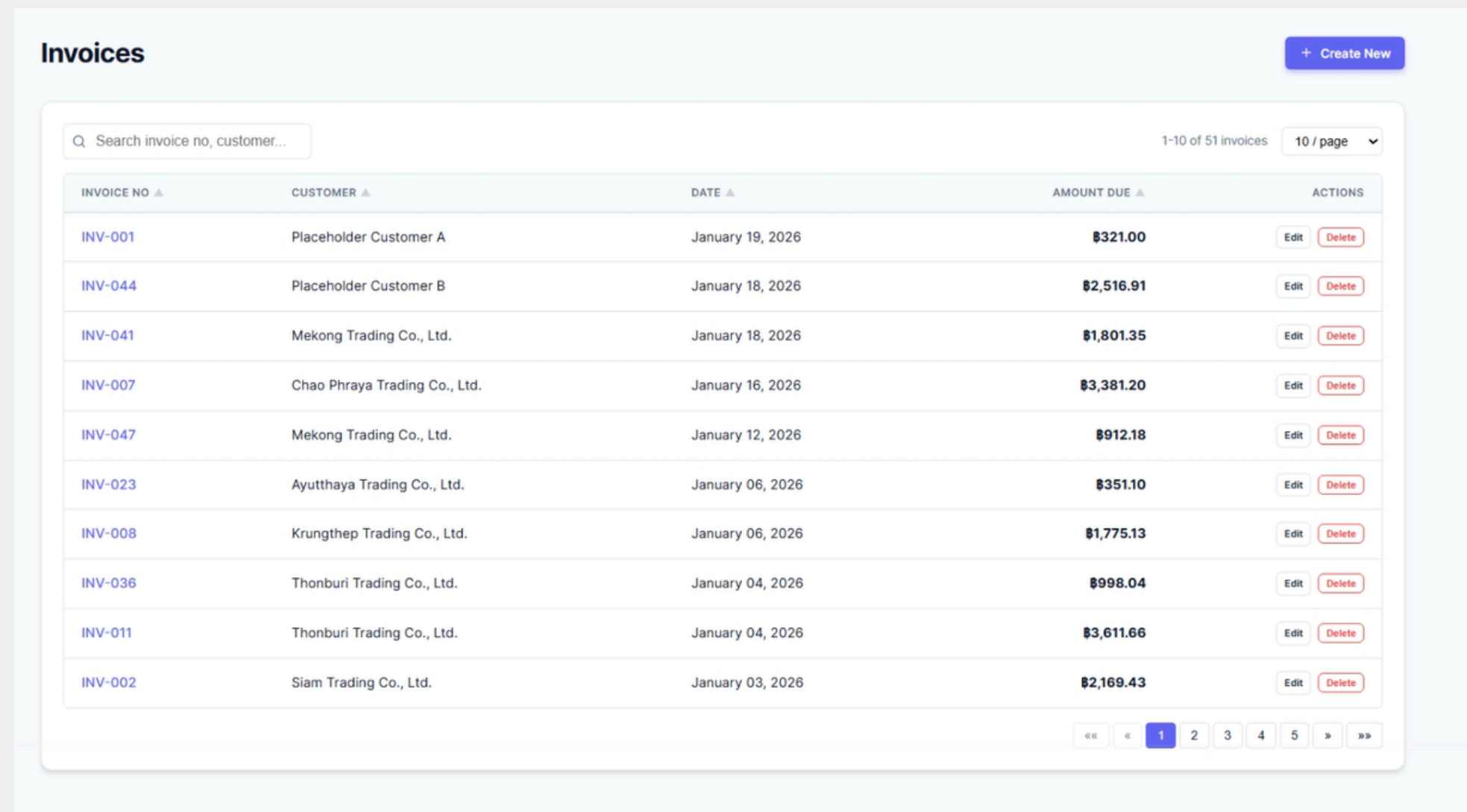
Important Files & Folder

File: client/src/components

- Store reusable UI components. Its would be in same pattern.



File

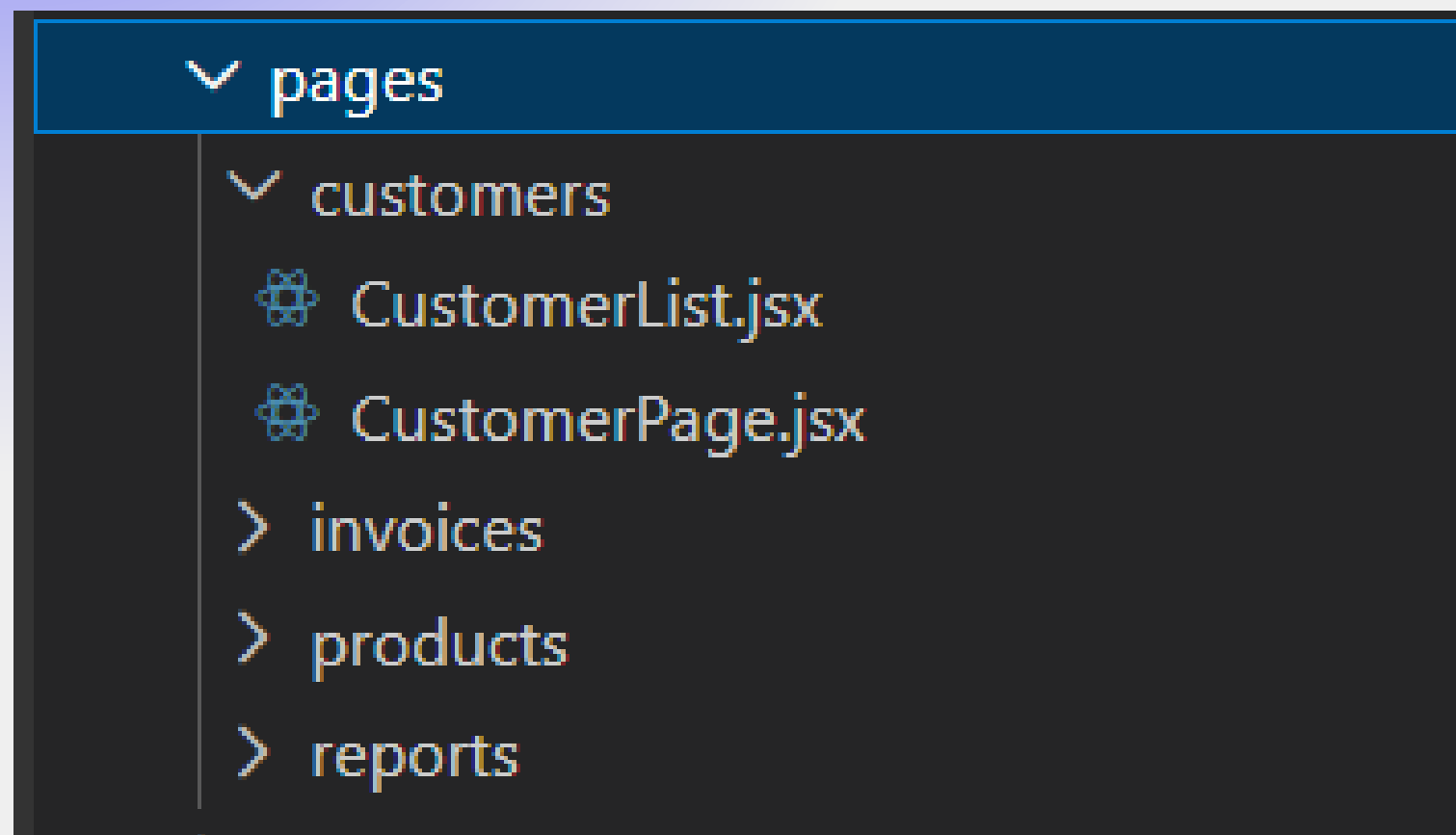


Example of DataList component

Important Files & Folder

File: client/src/pages

- Combine components, APIs, and logic to create pages



File

Example of Lists Page

File: client/src/pages/customers/CustomerList.jsx

```
export default function CustomerList() {
```

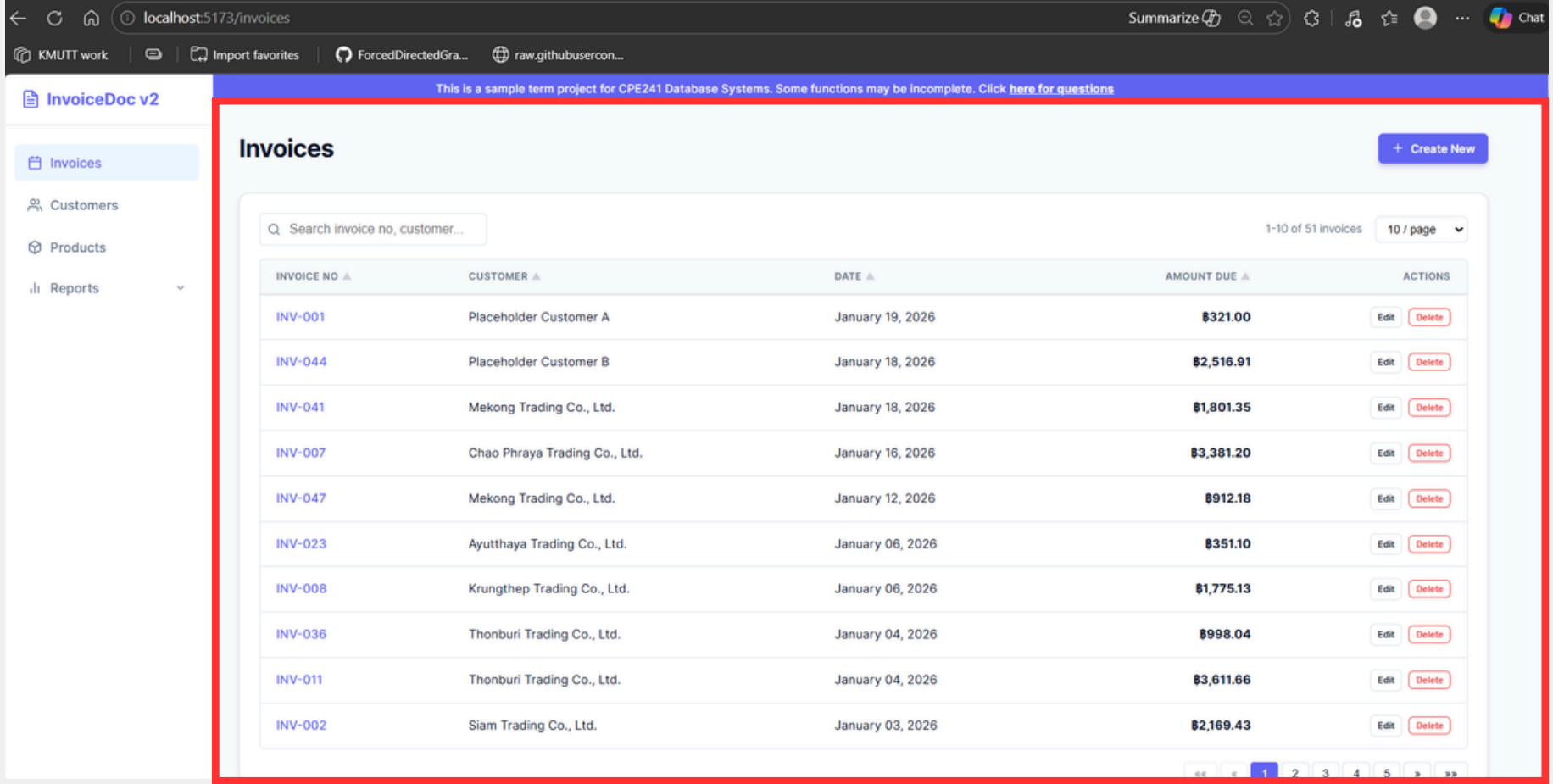
Use to call in main.jsx

```
return (  
  <>
```

Sets the interface in return section

```
<DataList  
  title="Customers"  
  fetchData={fetchData}  
  columns={columns}  
  searchPlaceholder="Search code, name, address..."  
  itemName="customers"  
  basePath="/customers"  
  itemKey="code"  
  onDelete={handleDelete}  
  refreshTrigger={refreshTrigger}  
/>
```

Call component



The screenshot shows a web application titled "InvoiceDoc v2" running on localhost:5173. The main content area displays a table of invoices. The table has columns for Invoice No., Customer, Date, Amount Due, and Actions. There are 10 invoices listed, each with an "Edit" and "Delete" button. A search bar is at the top of the table, and a "Create New" button is in the top right corner. The application is running on a browser with a dark theme.

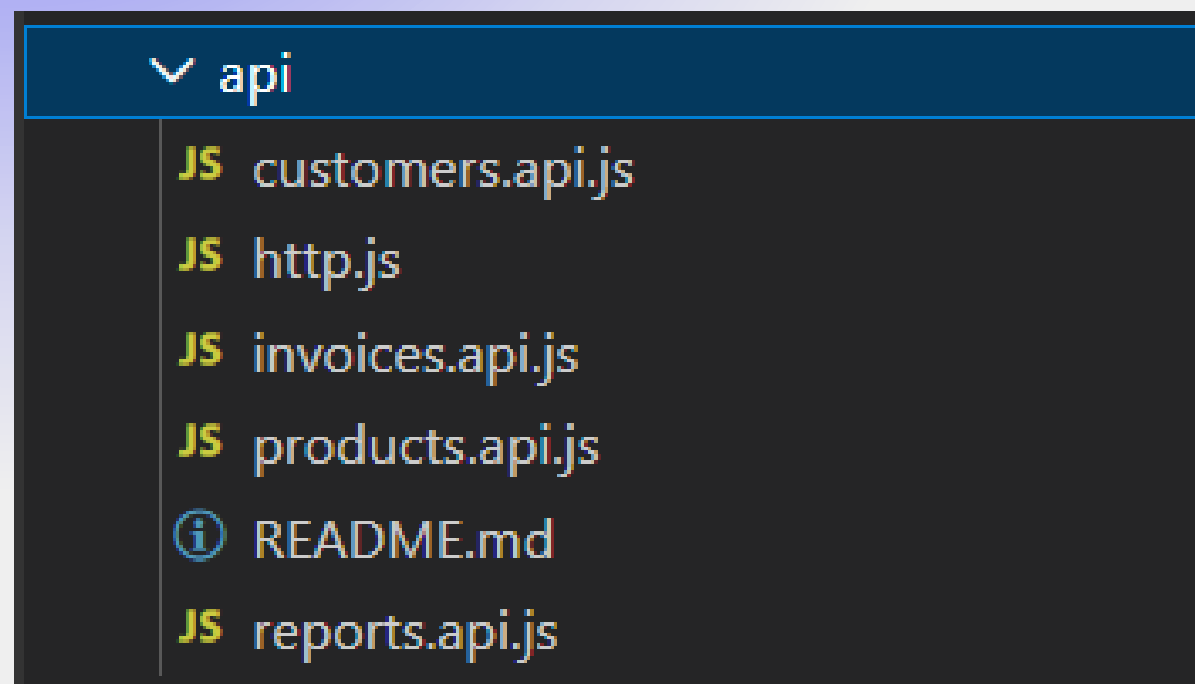
INVOICE NO	CUSTOMER	DATE	AMOUNT DUE	ACTIONS
INV-001	Placeholder Customer A	January 19, 2026	\$321.00	Edit Delete
INV-044	Placeholder Customer B	January 18, 2026	\$2,516.91	Edit Delete
INV-041	Mekong Trading Co., Ltd.	January 18, 2026	\$1,801.35	Edit Delete
INV-007	Chao Phraya Trading Co., Ltd.	January 16, 2026	\$3,381.20	Edit Delete
INV-047	Mekong Trading Co., Ltd.	January 12, 2026	\$912.18	Edit Delete
INV-023	Ayutthaya Trading Co., Ltd.	January 06, 2026	\$351.10	Edit Delete
INV-008	Krungthep Trading Co., Ltd.	January 06, 2026	\$1,775.13	Edit Delete
INV-036	Thonburi Trading Co., Ltd.	January 04, 2026	\$998.04	Edit Delete
INV-011	Thonburi Trading Co., Ltd.	January 04, 2026	\$3,611.66	Edit Delete
INV-002	Siam Trading Co., Ltd.	January 03, 2026	\$2,169.43	Edit Delete

System

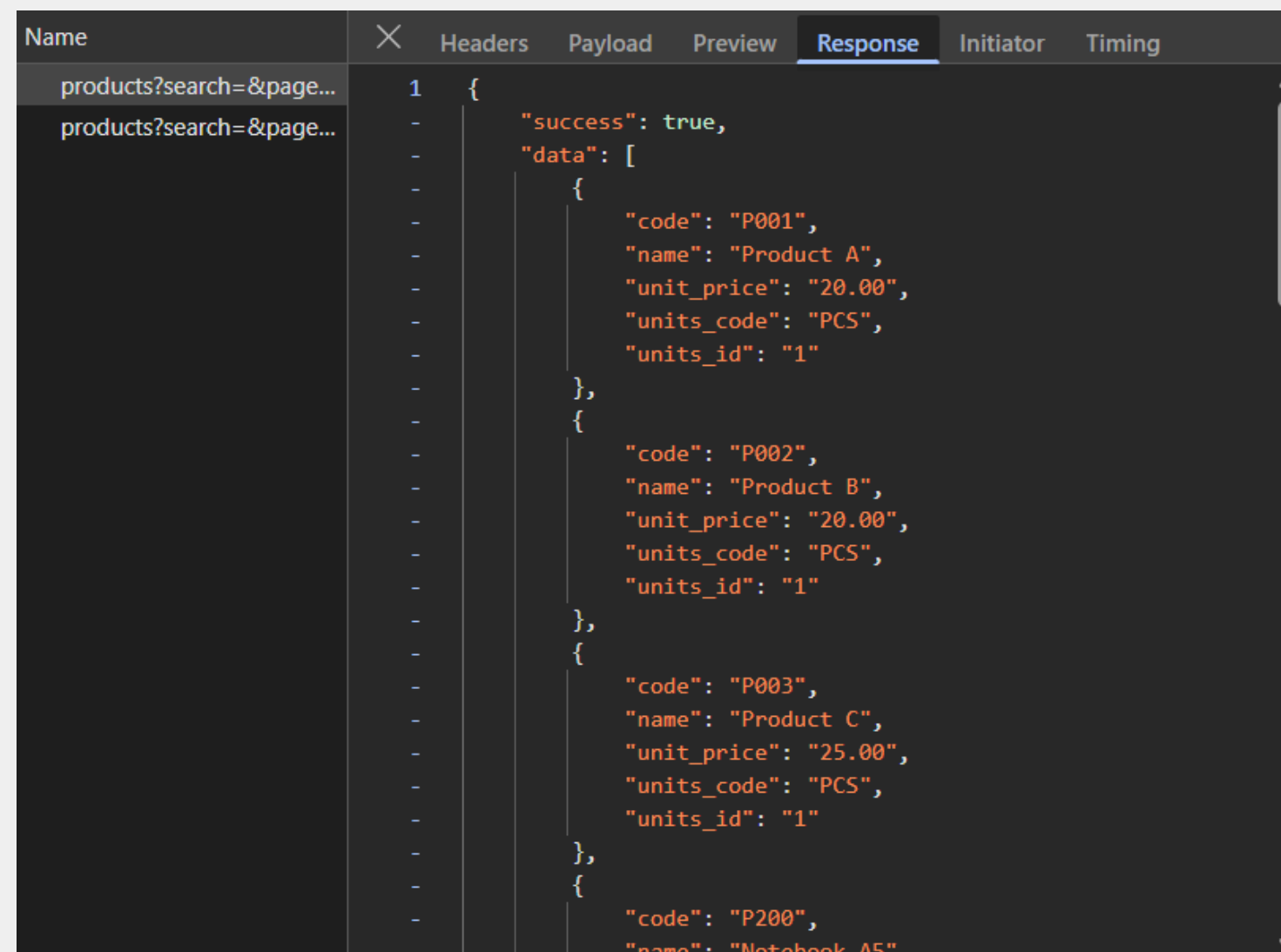
Important Files & Folder

File: client/src/api

- Set up and call APIs in the frontend



File



System

https

File: client/src/api/http.js

- Config API

```
// Backend API base URL (override via .env VITE_API_BASE)
export const API_BASE = import.meta.env.VITE_API_BASE !== undefined ? import.meta.env.VITE_API_BASE : "http://localhost:4000";

/**
 * Calls the backend API.
 * Response: { success, data, meta? } or { success: false, error: { message } }.
 * Throws Error if success is false or status is not 2xx.
 */
export async function http(path, options = {}) {
  const baseUrl = API_BASE.endsWith("/") ? API_BASE.slice(0, -1) : API_BASE;
  const res = await fetch(`${baseUrl}${path}`, {
    headers: { "Content-Type": "application/json", ...(options.headers || {}) },
    ...options,
  });
  const ct = res.headers.get("content-type") || "";
  const isJson = ct.includes("application/json");
  const body = isJson ? await res.json() : await res.text();

  if (!res.ok) {
    const message = isJson && body?.error?.message ? body.error.message : (typeof body === "string" ? body : `HTTP ${res.status}`);
    throw new Error(message);
  }
  return body;
}
```

→ Sets API base URL in .env or using default URL.

→ Sets ok or error response

Example of API

File: client/src/api/products.api.js

- Set how to call API



```
import { http } from "../http.js";

// If backend returns success: false, throw the error message.
function unwrap(res) {
  if (res && res.success === false && res.error) throw new Error(res.error.message);
  return res;
}

export async function listProducts(params = {}) {
  const query = new URLSearchParams(params).toString();
  const res = unwrap(await http(`/api/products${query} ? ` ?`${query}` · ""}`));
  return { data: res.data, ...(res.meta || {}) };
}
```

API endpoint

Name

products?search=&page...

products?search=&page...

Let's Try

Let's try adding ListPayment page with API and Route

1. Adding API

File: client/src/api/payments.api.js

```
import { http } from "../http";
```

To use http that we had set,
we need to import first.

```
export async function listPayments(params = {}) {
```

```
  const query = new URLSearchParams(params).toString();
```

```
  const res = await http(`/api/payments/list${query} ? `?${query}` : ""`);
```

```
  return res;
```

```
}
```

Sets API endpoint

2. Adding Page

File: client/src/pages/payments/PaymentList.jsx

```
import React from "react";
import DataList from "../../components/DataList";
import { listPayments } from "../../api/payments.api.js";
```

```
export default function PaymentsList() {
```

```
  const fetchData = React.useCallback((params) => listPayments(params), []);
```

```
  const columns = [
    { key: "id", label: "Payment No" },
    { key: "created_at", label: "Created At" },
    { key: "invoice_id", label: "Invoice ID" },
    { key: "amount", label: "Amount" }
  ];
```

```
  return (
    <DataList
      title="Payments"
      fetchData={fetchData}
      columns={columns}
      itemName="payment"
      emptySearch="No payments found."
      emptyDefault="No payments yet."
    />
  );
}
```

Payments

+ Create New

Q Search...

0 payment 10 / page

PAYMENT NO	CREATED AT	INVOICE ID	AMOUNT	ACTIONS
27	2026-02-02T04:01:00.000Z	21	4613.25	Edit Delete
26	2026-02-02T04:00:00.000Z	20	3158.75	Edit Delete

→ Call API

→ Sets column

→ Use Component

3. Adding Route

File: client/src/main.jsx

```
import PaymentsList from "../pages/payments/PaymentList.jsx";
```

Always import
before using!!!

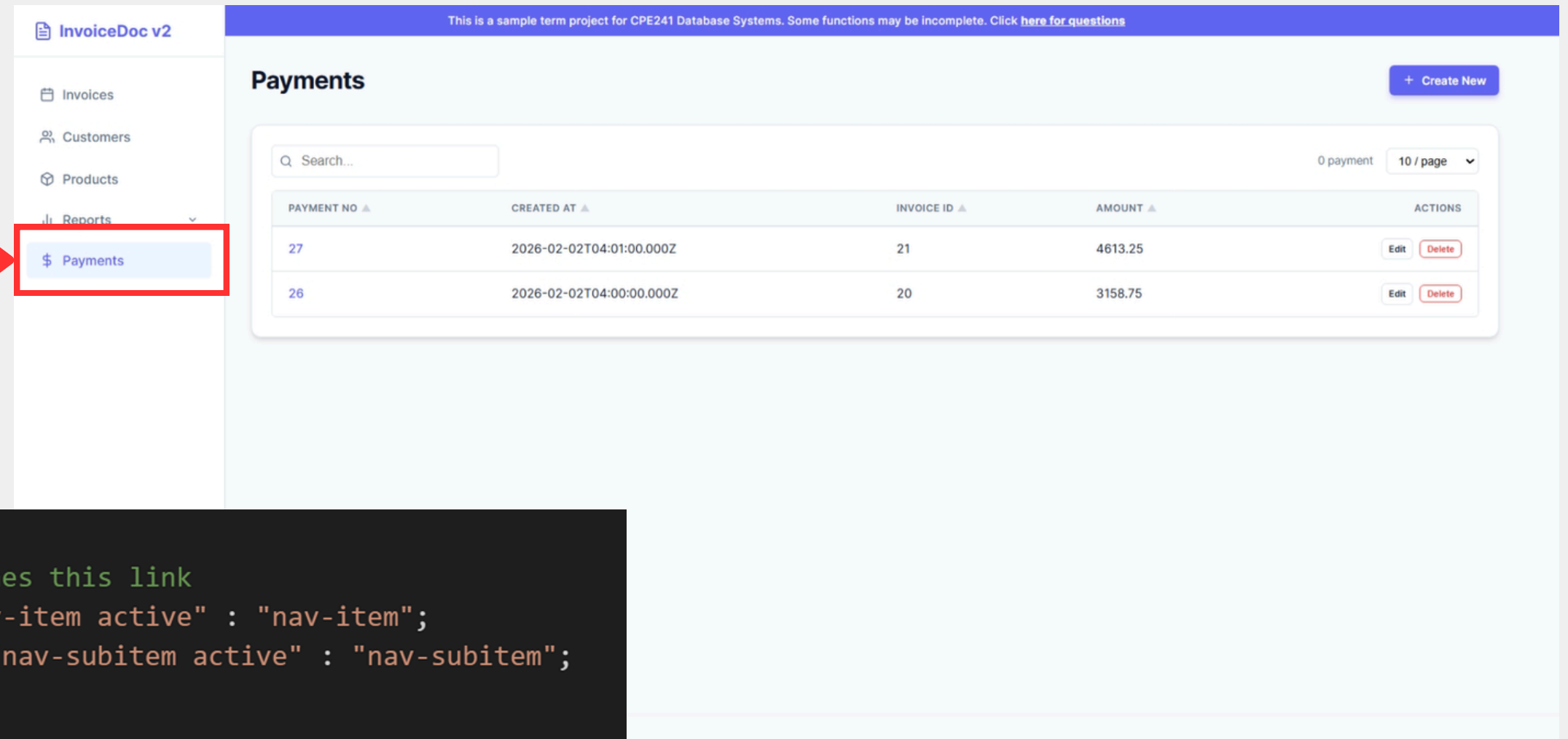
```
ReactDOM.createRoot(document.getElementById("root")).render(  
  <React.StrictMode>  
    <BrowserRouter>  
      <Routes>  
        {/* Default route → invoices list */}  
        <Route path="/" element={<Navigate to="/invoices" replace />} />  
        <Route path="/invoices" element={<Layout><InvoiceList /></Layout>} />  
        <Route path="/invoices/new" element={<Layout><InvoicePage mode="create" /></Layout>} />  
        <Route path="/invoices/:id" element={<Layout><InvoicePage mode="view" /></Layout>} />  
        <Route path="/invoices/:id/edit" element={<Layout><InvoicePage mode="edit" /></Layout>} />  
        <Route path="/customers" element={<Layout><CustomerList /></Layout>} />  
        <Route path="/customers/new" element={<Layout><CustomerPage mode="create" /></Layout>} />  
        <Route path="/customers/:id" element={<Layout><CustomerPage mode="view" /></Layout>} />  
        <Route path="/customers/:id/edit" element={<Layout><CustomerPage mode="edit" /></Layout>} />  
        <Route path="/products" element={<Layout><ProductList /></Layout>} />  
        <Route path="/products/new" element={<Layout><ProductPage mode="create" /></Layout>} />  
        <Route path="/products/:id" element={<Layout><ProductPage mode="view" /></Layout>} />  
        <Route path="/products/:id/edit" element={<Layout><ProductPage mode="edit" /></Layout>} />  
        <Route path="/reports" element={<Navigate to="/reports/product-sales" />} />  
        <Route path="/reports/product-sales" element={<Layout><Reports type="product-sales" /></Layout>} />  
        <Route path="/reports/monthly-sales" element={<Layout><Reports type="monthly-sales" /></Layout>} />  
        <Route path="/reports/customer-sales" element={<Layout><Reports type="customer-sales" /></Layout>} />  
        <Route path="/payments" element={<Layout><PaymentsList /></Layout>} />  
      </Routes>  
    </BrowserRouter>  
  </React.StrictMode>  
)
```

```
<Route path="/payments" element={<Layout><PaymentsList /></Layout>} />
```

Import new Route for PaymentList

4. Adding Path in Sidebar

File: client/src/main.jsx



```
function Sidebar() {  
  // NavLink injects isActive when the current URL matches this link  
  const getLinkClass = ({ isActive }) => isActive ? "nav-item active" : "nav-item";  
  const getSubLinkClass = ({ isActive }) => isActive ? "nav-subitem active" : "nav-subitem";  
  
  return (  

```

```
<NavLink to="/payments" className={getLinkClass}>  
  <svg style={{ marginRight: 10 }} width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round">  
    <path d="M12 1v22"></path><path d="M17 5H9.5a3.5 3.5 0 0 0 7H14a3.5 3.5 0 0 1 0 7H6">  
    </path>  
  </svg>  
  Payments  
</NavLink>
```

Set a page in the Sidebar

Now your ListPayment page is complete !!!
(But your new payment doesn't exits)